CEREBROHIVE™

Product Strategy and Execution Plan

A 24-week programme to convert the Enterprise Intelligence Mesh from architecture into a shippable, secure, revenue-bearing platform

Document	CerebroHive Product Strategy & 120-Day Execution Plan
Version	1.0 — Canonical
Date	28 July 2026
Prepared for	Board, investors, enterprise customers and partners, engineering leadership
Governing documents	CEREBROHIVE_CONSTITUTION.md · CAPABILITY_ARCHITECTURE.md · COMMERCIAL_STRATEGY.md
Evidence base	cerebro-hive-os monorepo, audit series, Milestone 25.4C/25.5 reports
Classification	Confidential — do not distribute without written consent

This document contains a candid technical assessment alongside forward-looking plans. Financial figures, timelines and hour estimates are planning estimates, not commitments, and are subject to the assumptions register in Appendix D.

Document Control

This plan is derived directly from the state of the CerebroHive monorepo as of 28 July 2026. Where it makes a claim about what exists today, that claim is traceable to a named file or audit report. Where it makes a claim about the future, it is labelled as a plan, an estimate or an assumption. The distinction is deliberate and load-bearing: the single most valuable property of this document is that a reader can tell the difference.

Version history

Version	Date	Author	Change
0.1	18 Jul 2026	Platform Engineering	Initial milestone framing (M10, ten epics)
0.5	24 Jul 2026	Platform Engineering	Superseded by phased M10.1–M10.7 backlog
0.9	27 Jul 2026	Architecture	Audit series complete; P0 auth gap escalated
1.0	28 Jul 2026	Programme Office	Canonical strategy and 24-week execution plan

Source documents consumed

- `CEREBROHIVE_CONSTITUTION.md` — vision, portfolio, governance standards, eight-phase roadmap
- `CAPABILITY_ARCHITECTURE.md` — five-tier model, dependency graph, boot order, degradation map
- `COMMERCIAL_STRATEGY.md` — editions, licensing philosophy, marketplace economics
- `AGENT-RUNTIME-BACKLOG.md` — the M10.1–M10.7 phased runtime programme
- `audit/P0-AUTH-AUTHZ-GAP.md` — end-to-end trace of the authentication gap and its partial remediation
- `audit/MILESTONE-25.5-PRODUCTION-READINESS.md` — production readiness across the 11 deployed services
- `audit/POLYGLOT-ARCHITECTURE-MAP.md` — the full six-language service estate and deployment status
- `audit/RESILIENCE-AUDIT.md` — per-service failure handling, confirmed real vs. confirmed mock
- `audit/scaffold-ranking.md` and `audit/executive-dashboard.md` — scaffold density and web readiness scoring
- `RUNTIME-VALIDATION-CHECKLIST.md` — the human-in-the-loop validation gate for M25

How to read this document

If you are…	Read
An investor or board member	Sections 1, 2, 12, 15 and the risk register in Section 11. Roughly 20 pages.
An enterprise customer or partner	Sections 1, 2, 13, 14, 15. Section 3 is optional but is the honest answer to 'is it real?'
An engineering leader	Sections 3 through 10 in full. Section 3 is the baseline everything else is built on.
An individual contributor	Sections 3, 6, 7, 9, 10 and Appendix B. Section 9 is your backlog.
A finance or operations partner	Sections 7, 8, 12 and Appendix D.

Table of Contents

Figures and Tables

Figures

Principal tables

1 Executive Summary

CerebroHive has built an unusually complete architecture and an unusually incomplete product. That sentence is the entire situation. The monorepo contains 88 packages, 10 applications and 33 services across six programming languages, governed by a constitution, a five-tier capability architecture, 45 product specifications and a commercial strategy with seven priced editions. It also contains a production API that reads the caller's identity from an HTTP header and trusts it, a multi-agent planner that returns the same hard-coded three-node plan regardless of what you ask it, and a policy engine wired into every request that has zero policies loaded.

Both halves of that are true at the same time, and the second half is not a reason to discount the first. The architecture is genuinely good. The AI gateway has real circuit breakers, per-tenant rate limiting, provider failover and cost tracking. The authentication package has real Keycloak JWKS verification and a 28-permission RBAC model — it simply was never imported by the service that needed it. The Python planning service uses LangGraph with a real self-correcting fix loop. The observability stack has five concrete Prometheus alert rules and a six-panel Grafana dashboard. This is not a company that does not know how to build; it is a company that built breadth before it built depth, and now has to invert that.

This document sets out a 24-week, 120-working-day programme to invert it. The programme is deliberately narrow. It does not attempt to advance all 50 products. It advances one vertical slice of the platform — identity, runtime, memory, orchestration, observability — from architecture to production-grade, and it takes three business applications to a genuine general-availability standard on top of that slice. Everything else is explicitly deferred and named as deferred.

1.1 The situation in one page

Dimension	Where we are today	Where week 24 leaves us
Identity	Tenant identity read from client-supplied headers; no auth library present in platform-api's dependency tree until this month's fix	Verified JWT on every public surface, workspace-to-tenant ownership enforced in the data layer, cross-tenant regression suite in CI
Agent runtime	Single-agent conversation path partially real; tool loop short-circuited; no conversation persistence	Real end-to-end agent execution with tools, persistence, retrieval-backed memory and multi-agent orchestration
Orchestration	Two independent agent systems (HiveSwarm in Python/Go, Platform Runtime in TypeScript) with no defined relationship	One declared system of record, one planner, one execution provider; the other is either retired or formally scoped as a distinct product
Deployment	Two Helm template generations define the same 11 services with incompatible values keys; both render	One template generation, one values schema, verified render diff in CI, no duplicate Kubernetes object identities
Product surface	Studio live in production with the repository's highest scaffold density (421 flagged files)	Studio consumes the platform through one SDK; scaffold density reduced to a published, tracked figure
Commercial	Seven editions priced; no metering instrumentation to bill against them	Hive Credits metered end to end; usage attributable to a verified tenant; design-partner contracts in place
Assurance	No cross-tenant tests; SOC 2 aspirational	Penetration test complete and remediated; SOC 2 evidence collection automated; eval gates blocking merges

1.2 What this programme delivers

Six milestones, one every four weeks. Each has a binary definition of done — it either passes or it does not, and there is no partial credit.

1. **MS1 (week 4) — Trustworthy front door.** No public endpoint accepts an unauthenticated request. Tenant identity comes from a verified token. One Helm template generation renders. A single agent completes a real model call end to end over HTTP with no mock in the path.
2. **MS2 (week 8) — Working agent.** Tools execute and their results feed back to the model. Conversations persist across process restarts. RBAC is enforced, not merely available. Cross-tenant access is proven impossible by test, not by assertion.
3. **MS3 (week 12) — Remembering agent.** Retrieval-backed memory answers correctly about facts stated beyond the raw context window. The HiveSwarm / Platform Runtime convergence decision is made and executed. CerebroSearch serves hybrid vector and keyword results.
4. **MS4 (week 16) — Coordinating agents.** A natural-language intent produces a genuinely different plan each time and each node's output comes from a real model call. CerebroFlow reaches feature-complete. Load and chaos testing begins.
5. **MS5 (week 20) — Sellable platform.** The public SDK is frozen and Studio consumes only that surface. Penetration testing is complete and findings remediated. Hive Credits metering is accurate to within 2% of provider invoices.
6. **MS6 (week 24) — Controlled general availability.** Three design-partner accounts in production. SOC 2 Type I evidence package assembled. Published SLOs with 30 days of measured attainment behind them.

1.3 Investment and expected return

The programme costs an estimated $5.83 million over 24 weeks, of which 67% is engineering payroll for a team ramping from 13 to 26 full-time equivalents. Infrastructure, inference and tooling together account for $1.04 million; security assurance and SOC 2 readiness account for $285,000; contingency is held at 12%.

Line	24-week cost	Basis
Engineering payroll (blended)	$3,880,000	17,440 hours at a blended $222/hour fully loaded, plus programme and product roles
AI and LLM inference	$468,000	Development, evaluation harness runs and three design-partner workloads
Cloud compute and storage	$396,000	Three environments, GPU allocation for embeddings, pgvector and object storage
Observability and tooling	$172,000	OTEL collector footprint, Grafana/Loki/Thanos, CI minutes, licences
Security assurance and SOC 2	$285,000	External penetration test, two remediation cycles, Type I readiness engagement
Contingency (12%)	$624,000	Held centrally against the risk register, released by milestone
Total	**$5,825,000**	

Return. Week 24 is the first point at which CerebroHive can defensibly sign an enterprise contract. Until identity is verifiable and tenants are provably isolated, every enterprise deal carries a diligence risk that no discount can offset — a single failed security review costs more in credibility than the entire security workstream costs to execute. The commercial model in `COMMERCIAL_STRATEGY.md` prices the Enterprise edition on a platform base plus metered Hive Credits; neither half of that model is billable today because usage cannot be attributed to a verified tenant. This programme makes the existing pricing model executable. On a conservative reading — three design partners converting at Business edition economics with modest credit consumption — the programme underwrites its own cost within the four quarters that follow it, and it removes the categorical blocker on every deal above that size.

> **THE HONEST FRAMING**
>
> This is not a growth programme. It is a debt-retirement and foundation programme with a commercial deadline attached. Anyone evaluating it should judge it on whether the week-24 state is genuinely sellable, not on how many of the 50 named products moved. The answer to "how many products advanced" is: five. That is the correct number.

1.4 The six decisions we are asking for

Execution is blocked on decisions, not on effort. Each of the following needs an owner and a date; four of the six are needed inside the first three weeks.

#	Decision	Needed by	Consequence of delay
D1	Confirm whether api.cerebrohive.com and gateway.cerebrohive.com currently require a token in the live cluster	Week 1	The severity of the P0 finding is unresolved; incident response cannot be scoped
D2	Choose one Helm template generation and delete the other	Week 2	Every infrastructure change lands twice with unpredictable precedence
D3	Declare the system of record: HiveSwarm, Platform Runtime, or a defined split	Week 4	Two teams build two planners; memory and tooling are duplicated
D4	Approve the service consolidation from 33 services to a target estate	Week 6	Operational burden scales with service count, not with delivered capability
D5	Approve the hiring plan and its budget envelope	Week 3	The ramp in Figure 5 slips week for week; MS4 onward becomes unachievable
D6	Select and contract three design partners	Week 5	The GTM track loses its feedback loop and beta readiness has no evidence base

1.5 What we are explicitly not doing

Programmes of this kind fail by addition. The following are named here so that a request to add them is visibly a change of scope rather than a clarification of it.

- No new products. The 45 product specifications remain specifications. Five products receive engineering investment; the other 45 receive none.
- No new *-core packages. Sixty-plus domain packages averaging roughly 350 lines already exist. The programme completes existing packages; it does not create peers for them.
- No vertical or industry SKUs. CerebroHealth, CerebroFinance and the rest belong to horizon three at the earliest.
- No marketplace. HiveExchange requires a stable SDK, a billing engine and a partner programme, in that order. The SDK lands at MS5; the rest follows this programme.
- No on-premises or air-gapped deployment. The Government edition is a horizon-two conversation and carries its own compliance programme.
- No model training. CerebroHive routes to frontier providers. Fine-tuning is a research activity, not a week-1-to-24 activity.

2 Vision and Product Strategy

CerebroHive exists to solve enterprise fragmentation. Organisations hold their data in disjointed systems, their automations in isolated tools and their expertise in individual heads. Each new AI product added to that estate makes the fragmentation worse, because each brings its own identity model, its own memory, its own audit trail and its own idea of what the organisation knows. CerebroHive's proposition is that intelligence should be an infrastructure layer — as fundamental to the enterprise as compute and networking — rather than a feature bolted onto thirty separate applications.

2.1 The Enterprise Intelligence Mesh

The product is an AI-native operating system that connects people, data, models, agents, workflows and infrastructure into a single intelligence layer. The architectural expression of that is a strict five-tier model in which every capability has exactly one owner and dependencies flow in one direction only.

Figure 1 — CerebroHive Five-Tier Platform Architecture

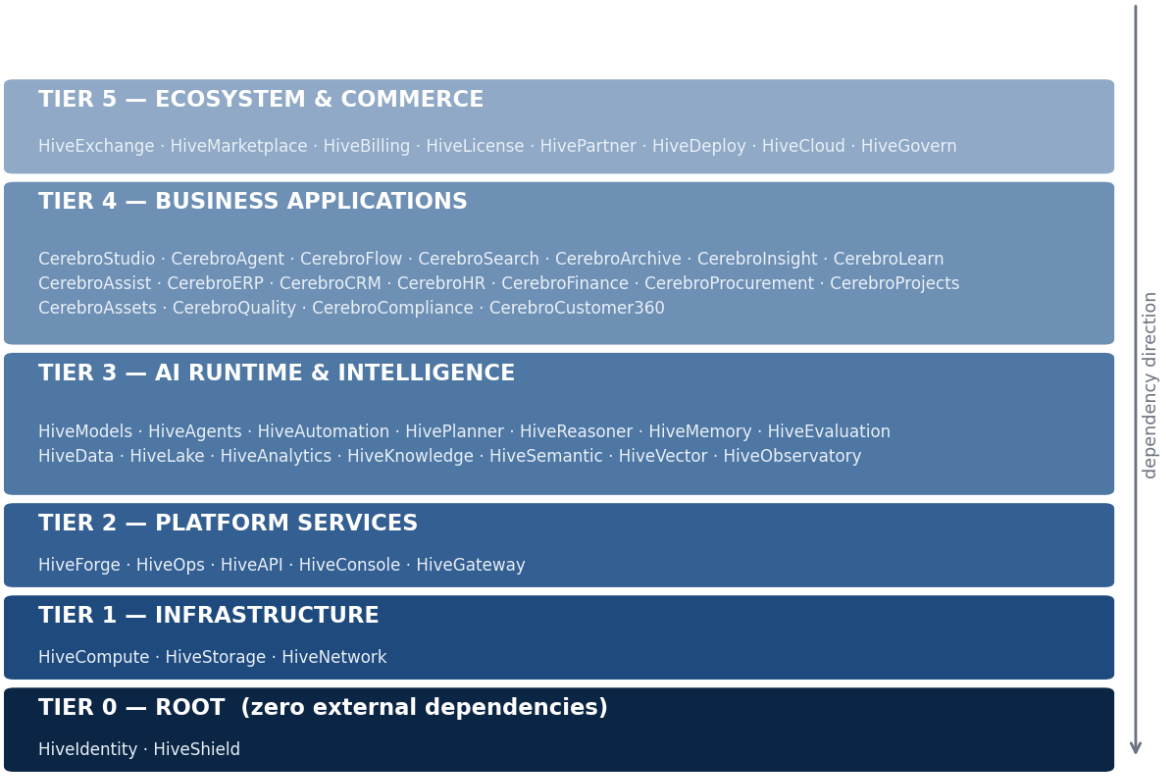

Figure 1 — The five-tier platform architecture. A product at tier N may depend on tier N−1 and below, never upward. Tier 0 has zero upstream dependencies and boots first.

The rule that makes this more than a diagram is the prohibition on upward dependency. It is what allows the platform to have a deterministic cold-start sequence — identity, then shield, then network, storage and compute, then the platform services, then the AI runtime, then applications — with a target full-platform boot of under 45 minutes. It

is also what makes graceful degradation specifiable rather than emergent: when the vector service fails, search falls back to keyword-only ranking; when the memory service fails, agents run session-only; when knowledge fails, vector search continues and graph traversal is disabled. Each of those fallbacks is a declared behaviour, not an accident.

2.2 Why the timing works

Three conditions have converged. Frontier model capability has crossed the threshold where multi-step tool-using agents are commercially useful rather than demonstrably fragile. Enterprise buyers have moved past pilot fatigue and now purchase against governance, auditability and cost attribution rather than against demo quality. And the first generation of AI point solutions has left large organisations with exactly the fragmentation problem CerebroHive is designed to absorb — which means the incumbent is not a competitor, it is the customer's own sprawl.

The corollary is uncomfortable and worth stating plainly: buyers who evaluate on governance and auditability will discover an unauthenticated API in the first hour of technical diligence. The market timing is favourable precisely along the axis where the current implementation is weakest. That is the strategic reason the security workstream is sequenced first rather than the compliance reason.

2.3 Portfolio and sequencing

The portfolio divides into Cerebro applications, which face the business, and the Hive platform, which faces developers and operators. The constitution defines a strict engineering dependency flow — HiveForge, then HiveOps, then HiveAPI, then CerebroStudio, then CerebroFlow, then CerebroAgent, then CerebroERP, then industry solutions — and this programme respects it.

Product	Role	Status entering the programme	Programme treatment
HiveIdentity	Tier 0 — identity for humans and agents	Library complete, unwired	Wired into every public surface (WS1)
HiveShield	Tier 0 — AI-specific security, DLP	Ingress WAF real; DLP absent	Policy enforcement activated (WS1/WS3)
HiveGateway	Tier 2 — traffic, auth, rate limiting	Rust gateway real but bypassed by ingress	Route decision, then single enforced path
HiveAgents / HivePlanner	Tier 3 — agent runtime and planning	Split across two implementations	Converged; one real planner (WS2/WS3)
HiveMemory / HiveVector	Tier 3 — memory and retrieval	Fully modelled, unpopulated	Repositories and retrieval built (WS2)
CerebroStudio	Tier 4 — unified command centre	Live, highest scaffold density	Audited, re-platformed onto the SDK (WS4)
CerebroFlow	Tier 4 — automation and pipelines	Partial	Taken to GA (WS4)
CerebroSearch	Tier 4 — semantic search	Modelled	Hybrid vector + BM25 delivered (WS4)
All other products	Tiers 3–5	Specified	No engineering investment this programme

2.4 Strategic moats

Four of the ten moats named in the constitution are defensible in the medium term; the rest are aspirations that become moats only after the platform is real. Being clear about which is which is what makes the strategy actionable.

Moat	Assessment
Unified knowledge graph and ontology	**Genuine and durable.** Once an enterprise's entities, relationships and access boundaries are modelled inside CerebroHive, migrating that graph elsewhere is a multi-quarter project. This is the strongest moat in the portfolio and it is currently the least built.
Multi-agent orchestration at scale	**Genuine but contested.** The LangGraph planner in services/planner-service is more sophisticated than most commercial equivalents. The moat is real; the risk is that it is duplicated internally rather than deepened.
Governance and compliance depth	**Genuine and increasingly decisive.** Immutable audit, control-to-evidence mapping and per-action lineage are what enterprise buyers actually cannot build themselves under time pressure.
Cloud-agnostic, modular deployment	**Genuine but expensive.** It is a real differentiator against hyperscaler-native offerings, and it is the direct cause of the polyglot operational burden in Section 3.6. It should be a deliberate choice, not a default.
Specialised models rather than one monolith	**Not yet a moat.** Routing between frontier providers is a capability, not a barrier. It becomes a moat only when routing decisions are informed by proprietary evaluation data — which requires the eval harness in WS5.
Marketplace ecosystem	**Not yet a moat.** Marketplaces are moats at liquidity, not at launch. Correctly deferred beyond this programme.

2.5 Commercial model

The pricing philosophy rejects per-seat licensing, and it is right to. If agents do the work of people, charging per seat penalises the customer for adopting the product's core value. The model is instead a platform base plus metered consumption: a flat annual fee per edition granting access to the software, governance tooling and support; Hive Credits burned against inference tokens, agent compute minutes, vector storage and completed workflow tasks; and seat licences charged only for authors and administrators, with consumers included at no cost.

This model is coherent, defensible and currently unbillable. Metering requires that consumption be attributable to a verified tenant, and tenant identity today comes from a header the caller sets. Section 12 treats metering instrumentation as a first-class deliverable of this programme rather than as a billing afterthought, for exactly that reason.

Edition	Target buyer	Deployment	Programme relevance
Starter	Startups and small teams	Shared cloud, self-serve	Not addressed; requires self-serve signup
Professional	Mid-market	Shared cloud	Reachable at MS6 given metering
Business	Enterprise departments	Dedicated tenant	**Primary target of this programme**
Enterprise	Fortune 500	Private cloud / VPC	Reachable in the two quarters after MS6
Enterprise Plus	Regulated or very large scale	Multi-cloud, federated	Horizon two
Government / Defence	Federal, DoD	Air-gapped, IL5/IL6	Horizon two; separate compliance programme
OEM / Embedded	ISVs	API-driven	Requires the frozen SDK from MS5

2.6 Maturity model as a sales instrument

The seven-level capability maturity model — assistant, copilot, autonomous agent, multi-agent team, department automation, intelligence mesh, autonomous enterprise — is the most commercially useful artefact in the

constitution, because it lets a buyer locate themselves and see the next step rather than being asked to buy a decade of vision at once.

It also imposes an honesty discipline internally. At the start of this programme CerebroHive can credibly deliver level 1 and, in the HiveSwarm path, parts of level 3. It cannot credibly deliver level 4 today, because multi-agent coordination runs against a planner that returns a fixed plan. At MS4 the platform delivers level 4 for real. Sales positioning should track that line and not lead it — a level-6 pitch against a level-1 implementation is the fastest way to lose a reference account permanently.

Figure 8 — Product Roadmap Horizon

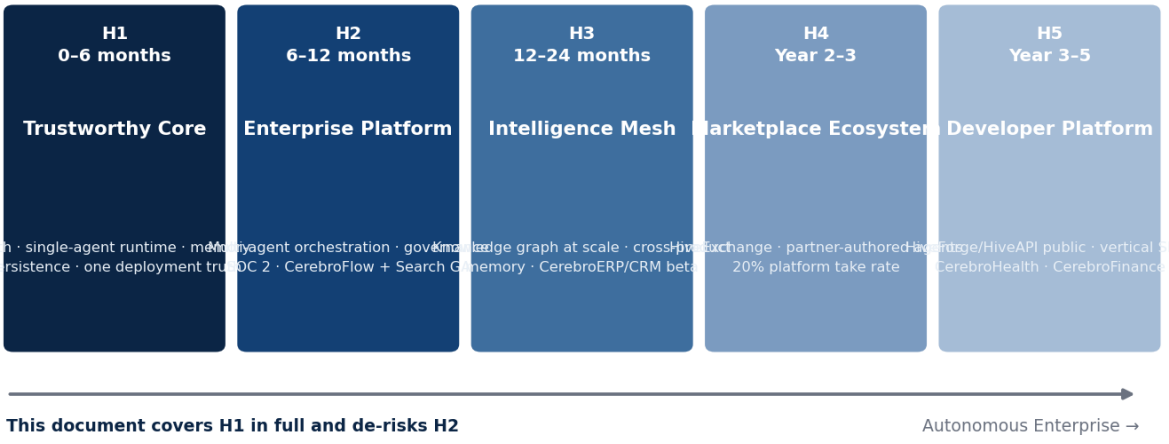

Figure 8 — Product roadmap horizon. This document covers horizon one in full detail and removes the principal technical risks from horizon two.

3 Current Architecture Assessment

This section is written to be uncomfortable to read and expensive to ignore. Every finding below is traceable to a named file or an audit report in the repository. Where something is confirmed, it says confirmed and gives the evidence. Where something is inferred from structure without reading business logic line by line, it says so. Where something genuinely cannot be determined from the repository alone — because it depends on the live cluster — it is listed as unresolved rather than guessed at.

3.1 Method and evidence base

Three independent methods were used, and findings are reported at the confidence level the weakest supporting method allows.

- **Source reading.** Business logic read directly. Highest confidence. Used for the AI gateway, the auth package, platform-api's middleware and routes, the swarm execution engine and planner, the Rust gateway and the Go tool gateway.
- **Structural confirmation.** Framework and file layout confirmed real — a Spring Boot controller/service/repository tree, for instance — without reading the logic inside. Used for the three Kotlin services and the C++ ML service.
- **Inventory and static analysis.** File counts, dependency manifests and regex scanning for scaffold markers (TODO, FIXME, Mock*, Stub, placeholder, NotImplemented). Lowest confidence; flags candidates rather than proving anything.

> **A CAVEAT ON SCAFFOLD COUNTS**
>
> A scaffold hit is a regex match, not a verdict. Prisma's generated client accounts for most of packages/db's hits, and platform-api's hits include legitimate test doubles from deliberate work. The count is a search heuristic that tells you where to look, not a measure of quality. It is used that way throughout this section.

3.2 Repository topography

The monorepo is a pnpm and Turborepo workspace on Node 22, with a Next.js web tier, Fastify and NestJS services, and additional services in Python, Go, Kotlin, Rust and C++. Code volume tells the story more clearly than the package count does.

Table 3.1 — Code volume by workspace

Workspace	Units	Observation
packages/	88 packages	Two packages exceed 2,000 lines of hand-written source (runtime-core at ~2,770 and swarm-sdk at ~2,056). Roughly sixty average 300–450 lines. packages/db's 51,000 lines are almost entirely generated Prisma client.
apps/	10 applications	studio (1,163 files, live in production), forge, platform, platform-api, plus archive, flow, insight, ops, pulse, search at varying maturity.
services/	33 services	**Sixteen contain zero TypeScript source.** Of the seventeen with code, four exceed 1,000 lines (forge-api ~2,489, platform-api ~2,158, contentops ~1,009, temporal-worker ~694). Non-TypeScript services — Python, Go, Kotlin, Rust, C++ — are not reflected in these line counts and several are substantial.

Workspace	Units	Observation
infra/	23 domains	Terraform, Helm, ArgoCD, Prometheus, Thanos, Velero, KEDA, chaos, k6, OPA policy, flagd. Unusually complete for the stage.
docs/ and specs	45 product specs, 12 doc domains	The strategic and specification layer is the most complete part of the repository by a wide margin.
CI	14 workflows	Includes a policy gate, an LLMOps evaluation gate, CodeQL, Lighthouse, load testing, a release train and self-healing workflows.

The headline ratio. Eighty-eight packages and thirty-three services against roughly a dozen subsystems with more than a thousand lines of hand-written logic. The architecture has been laid out at full scale and populated at pilot scale. This is not inherently wrong — laying out the tier boundaries early is why the dependency graph is clean — but it does mean that package count, service count and product count are all misleading proxies for capability, and should not be used as progress metrics by anyone, internally or externally.

3.3 What is genuinely production-grade

Six subsystems are real, well-engineered and worth defending. Each was confirmed by reading source.

- `packages/ai-gateway` — **the strongest code in the repository.** Real Anthropic and OpenAI SDK calls; a per-provider circuit breaker with configurable error threshold and reset timeout; a per-organisation-and-provider rate limiter; provider failover ordered by priority; a response cache; cost tracking; and a retryable-versus-fatal error distinction. It also already calls telemetry hooks and already supports streaming.
- `packages/auth` — real Keycloak JWKS verification using `jose`; a full RBAC permission system with five organisation roles and roughly 28 fine-grained permissions; and a reference middleware showing the intended require-auth, require-role, require-permission and require-org-access pattern. **It was unused not because it was incomplete, but because the service that needed it never declared it as a dependency.**
- `services/forge-api` — uses the auth package correctly. Its NestJS guard extracts the bearer token, verifies it, throws on failure and populates a rich request context. This is the reference implementation the rest of the estate should have copied.
- `services/planner-service` — Python and LangGraph, with a real self-correction loop capped at two fix retries and cycle detection before plan execution. More sophisticated than anything in the TypeScript runtime.
- Observability — a Prometheus rule set with five concrete alerts including service-down, error-rate, p99 latency, swarm queue backlog and LLM token budget; a ServiceMonitor scraping every labelled pod every thirty seconds; and a six-panel Grafana dashboard against Prometheus and Loki. OTEL exporters are wired at the platform level with log level correctly gated per environment.
- Network policy — default-deny ingress with explicit allows for the ingress namespace, intra-platform pod traffic, DNS, outbound 443 for provider APIs, and Prometheus scraping. A legitimate zero-trust baseline at the network layer.

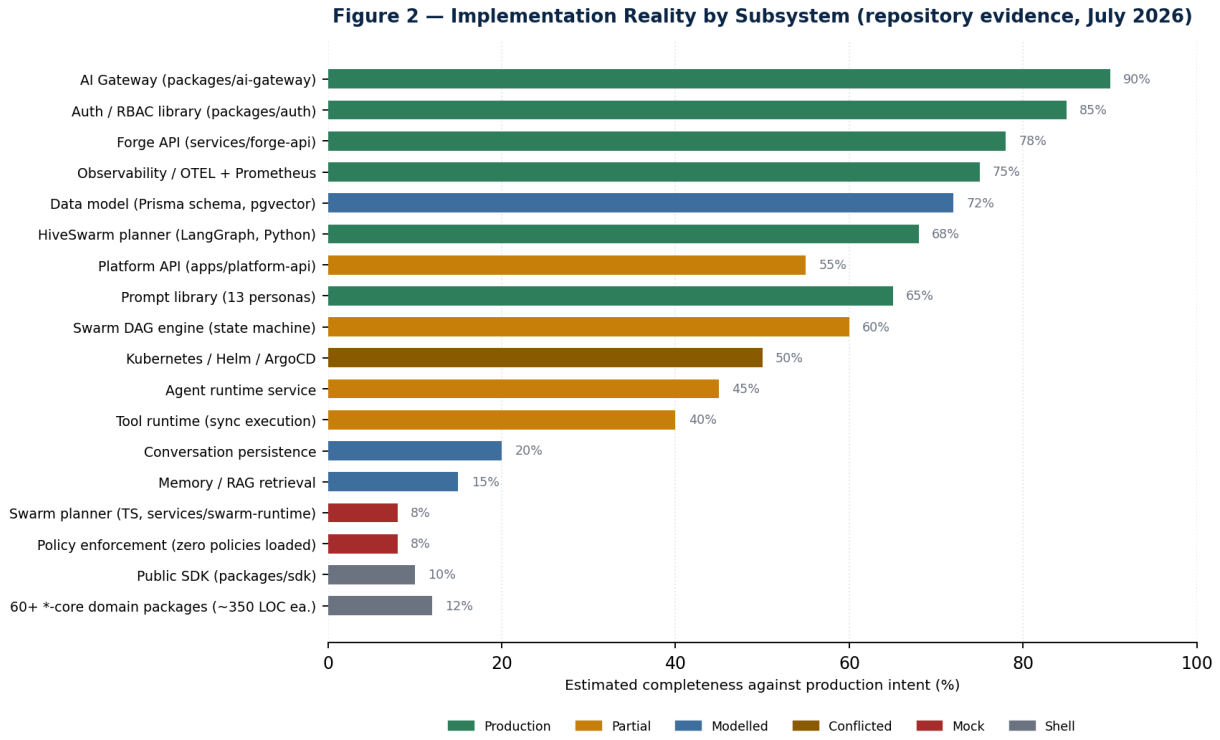

Figure 2 — Implementation reality by subsystem. Percentages are engineering judgement against production intent, anchored to the evidence in the audit series.

3.4 What is modelled but not wired

This is the most valuable category in the repository, and the one most likely to be misread as missing work. The Prisma schema already defines `AgentConversation`, `AgentMessage`, `AgentExecution`, `Memory`, `MemorySnapshot`, `Document`, `DocumentVersion`, `Chunk`, `Embedding` with a pgvector `vector(1536)` column, and `VectorIndex`. The migration is applied; the tables exist. The entire retrieval-augmented generation data layer is designed and live and contains nothing.

The implication for planning is significant and favourable. Conversation persistence and memory are not schema-design projects — they are repository-and-wiring projects, which are faster, lower-risk and far easier to estimate. The corresponding packages are also already reserved and already declared as dependencies in the right places. The correct instruction to the team is emphatic: build the repositories, do not re-model the schema.

> **WARNING CARRIED FORWARD FROM THE BACKLOG**
>
> The `stage2_sessions` migration created `AssessmentSession` and `SessionTelemetryBatch` tables. These belong to a talent-assessment feature area, not to agent conversations. Do not confuse them with `AgentConversation` and do not reuse them.

3.5 What is scaffolding, confirmed

Four items below are not inferred from regex — the code documents its own incompleteness in comments or in obviously placeholder behaviour.

Component	Confirmed state	Consequence
`services/swarm-runtime/Planner.ts`	compile() returns the same fixed three-node DAG for "Analyze Q3 spending" regardless of the input intent. Not exported from the package barrel.	Dead code that reads as a competing planner implementation. Resolves the open question in the architecture map: it is a mock, not a rival.
`ExecutionEngine` 's worker provider	The only wired execution provider simulates work with a hard-coded 800ms timeout and fails on a magic string. Retry policy explicitly not implemented — the code's own comment says so. Artifact storage commented as mock.	The DAG scheduler itself is genuinely good — topological dispatch, priority queue, worker-pool capacity, cancellation, failure cascade. Only the thing it dispatches to is fake.
`PolicyEngine` in platform-api	Constructed and wired into the request path with zero registered policies.	**Silently enforces nothing.** This is worse than absent, because the architecture diagram and the request path both suggest enforcement exists.
`packages/ai-gateway` container	`src/index.ts` — the file the Dockerfile actually runs — is a pure barrel export. No file in the package calls .listen() or constructs an HTTP server.	The standalone Kubernetes deployment executes some import statements and exits. Its own ingress at gateway.cerebrohive.com cannot be serving anything.
`services/gateway` root proxy route	Hard-coded empty base URL instead of the captured value. Carried over unfixed from an earlier audit.	The Rust gateway's root proxy is broken, though production ingress appears to bypass the gateway entirely.

Scaffold density, for triage only. apps/studio carries 421 flagged files out of 1,163 — the highest in the repository, and it is live in production, which raises the stakes on the audit rather than lowering them. apps/platform is next at 33, then platform-api at 22. Sixty-plus `*-core` packages carry between one and ten hits each against roughly 350 lines apiece, which is the signature of a scaffolded domain package rather than an incomplete one.

3.6 The polyglot service estate

The most consequential discovery in the audit series is that a scanner limited to TypeScript materially undercounted the system. There is a real, coherent, well-engineered multi-agent product spanning Python, Go and TypeScript, internally named HiveSwarm, running alongside the TypeScript platform runtime.

Table 3.3 — Service estate and deployment status

Service	Language	Deployed?	Status
services/gateway	Rust / Axum	CI-built, Helm-absent	Real, mostly working; one placeholder bug; production ingress appears to bypass it
packages/ai-gateway	TypeScript	Prod + staging	Real library; the deployed container has no server to run
apps/platform-api	TypeScript / Fastify	Prod + staging	Real; direct ingress at api.cerebrohive.com
services/forge-api	TypeScript / NestJS	Prod + staging	Real; correct auth; internal only
services/agent-runner	Python / LangChain	Prod + staging	Real, mature multi-agent orchestrator (HiveSwarm)
services/planner-service	Python / LangGraph	Prod + staging	Real, sophisticated goal-to-DAG planner (HiveSwarm)
services/swarm-api	Go + NATS + Redis	Prod + staging	Real and substantial; other HiveSwarm services depend on it
services/tool-gateway	Go / Gin	Prod + staging	Real, well built; serves HiveSwarm, not platform-api
services/router-service	Go	CI-built, Helm-absent	Real weighted agent-selection scorer; undeployed
services/memory-service	Python	Prod + staging	Deployed; source unread
services/swarm-runtime	TypeScript	Prod + staging	Deployed; contains the confirmed mock planner

Service	Language	Deployed?	Status
services/platform-svc, academy-svc, crm-svc	Kotlin / Spring Boot	CI-built, Helm-absent	Real Spring services; logic unread
services/ml-svc	C++ / gRPC	CI-built, Helm-absent	Real: embeddings, lead scoring, recommendations, pgvector
services/evaluation-service, learning-service	Python	CI-built, Helm-absent	Real; undeployed
services/temporal-worker	TypeScript	Not in the CI build matrix	Real Temporal worker with no confirmed consumer or deploy path
apps/studio, apps/forge	TypeScript / Next.js	Prod + staging	Live; studio carries the highest scaffold density in the repository

Eight services are built by CI and absent from the Helm chart. The repository contains no evidence either way as to whether they deploy through some mechanism outside it, or simply are not deployed anywhere. That includes the Rust gateway that the architecture positions as the edge, and all three Kotlin business services. This is a governance gap as much as a technical one — the repository does not know what is running.

3.7 Critical finding 1 — authentication and tenant isolation

> **P0 — CONFIRMED AT REPOSITORY LEVEL, UNRESOLVED AGAINST THE LIVE CLUSTER**
>
> The middleware that populates the request context — the single function every route trusts as ground truth for who is asking — read `tenantId`, `workspaceId` and `userId` directly and exclusively from client-supplied headers, with an in-source comment stating that this is demonstration code standing in for real authentication.
>
> Every downstream tenant-isolation check was therefore validating the fake context against itself. Checks of the form `agent.workspaceId !== workspaceId` are correct logic over an untrustworthy input. Setting a different workspace header passes every one of them. No privilege escalation is required; it is the default read path.

Why this is worse than simply missing authentication. A service with no authorisation layer is obviously unsafe and gets treated accordingly. A service with a look-alike authorisation layer that provides no protection is worse, because reviewers, architects and diagrams all report that isolation exists. The shared ingress that fronts studio, forge, platform-api and ai-gateway has real WAF with the OWASP core rule set, real rate limiting, real TLS and real security headers — and no authentication annotation of any kind. None of the real controls is authentication.

Remediation already implemented

Code only, not yet deployed or verified in the cluster. The auth package turned out to contain everything the recommended stack asked for; this was a wiring gap, not a missing-capability gap. The changes made:

- platform-api now declares `@cerebro/auth` as a dependency.
- A new Fastify preHandler mirrors forge-api's guard: extracts the bearer token, verifies it, returns 401 on missing, invalid or expired tokens, and on success overwrites tenant, user, role and permission context from the verified token claims.
- The header-based mock was stripped from the request-context middleware; trace and correlation IDs still come from headers, which is fine because they are not a security boundary.
- Route registration was restructured so the auth hook applies to a properly encapsulated child context. This is mechanical, not cosmetic: Fastify root hooks cascade to everything, so health checks would have started requiring a token and Kubernetes would never have marked the pod ready.

- A hard-coded port that ignored the injected `PORT` environment variable was fixed. The Helm chart injects 4000; the process listened on 3000. Readiness probes against 4000 would never have passed, independent of authentication.

What remains open — stated rather than quietly closed

- **Workspace ownership.** The token proves organisation membership, not which workspace inside that organisation. Workspace ID is still read from a header, so nothing yet confirms the authenticated tenant owns the workspace named. Closing this needs a workspace-to-tenant ownership lookup in the data layer. Scheduled at week 3.
- **The AI gateway.** "Add JWT middleware to ai-gateway" is not a well-defined task, because there is no server to add it to. That is a build-a-service decision, not a port-a-pattern decision.
- **RBAC enforcement.** The permission machinery exists and is not called from any route. Authentication without authorisation is half the control.
- **Audit logging.** An audit logger is constructed and passed into the application service; whether it is invoked anywhere, and what identity it records, has not been verified.
- **Tests.** No coverage of the authenticated, unauthenticated, invalid-token and cross-tenant scenarios. Until those exist as a permanent regression suite, the fix is a point-in-time claim.

3.8 Critical finding 2 — the Helm chart defines every service twice

The chart contains two template generations that both render into the same release, defining the same Kubernetes objects with the same names in the same namespace, keyed off two different values schemas.

	Generation 1	Generation 2
Files	service.yaml, deployment.yaml, rollout.yaml	deployments.yaml (a reusable macro despite the filename)
Values key style	snake_case (ai_gateway)	camelCase (aiGateway) — matches values.yaml's actual structure
Services covered	5	All 11 deployed services
Gating	Gated on rollouts.enabled being false	**Ungated. Always renders.**
PORT injection	Yes, and containerPort and probes match it	**No PORT env var at all**
Maturity	Higher — db-migrate initContainer, second secret mount, topology spread, pod anti-affinity	Lower, but broader coverage
Renders Ingress	Shared ingress.yaml	Its own per-service ingress at the same hostname

The production consequence. Production sets `rollouts.enabled: true`, which switches off generation 1's Deployment. Nothing switches off generation 2. So in production specifically there is a real possibility of an Argo Rollout and a plain Deployment both trying to own pods with the same labels, and of two Ingress objects claiming the same hostname. Which one wins depends on Helm's rendering order and on how the cluster reconciles identical object identities — determinable only by rendering the chart against the live values, not from the repository.

A second-order consequence worth its own mention. The AI provider secret is mounted only by generation 1, and only for ai-gateway and forge-api. platform-api never receives it under either generation. Since platform-api is the process that actually constructs the gateway in-process, and the gateway factory only enables a provider when the corresponding API key is present in the environment, this is a plausible functional failure independent of everything else: the service that needs the keys may not be receiving them.

3.9 Critical finding 3 — two agent runtimes, no declared relationship

HiveSwarm — Python agent-runner, Python LangGraph planner, Go swarm API, Go router, Go tool gateway — is a real multi-agent orchestration product. The platform runtime — platform-api plus the agent-builder capability plus the TypeScript ai-gateway — is a real single-agent conversation product. Both are deployed. Neither appears in the other's route table. The tool gateway serves HiveSwarm, not platform-api. platform-api does not proxy to the AI gateway over the network; it imports it in-process.

These are probably different products rather than competing implementations — multi-agent orchestrated automation on one side, conversational agents on the other. That is a reasonable hypothesis and it is not a decision. Until someone declares which is the system of record, both will grow their own planner, their own memory, their own tool catalogue and their own identity model, and the cost of merging them will compound every sprint. Decision D3 in Section 1.4 exists for this, and it is dated week 4 because the memory work in M10.5 is the last point at which the two can still diverge cheaply.

3.10 Critical finding 4 — resilience is real in places and absent in the places that matter

- **Confirmed real:** the AI gateway's circuit breaker, rate limiter, failover ordering and retryable-error classification; the Go tool gateway's Redis retry backoff and graceful shutdown; the LangGraph planner's capped self-correction loop and cycle detection; the Go router's hard cost, latency and proficiency ceilings; the Rust gateway's JWT and rate-limiting middleware.
- **Confirmed absent:** retry policy in the swarm execution engine, which the code itself annotates as failing immediately for simplicity; policy enforcement in platform-api, which runs with an empty policy set; artifact storage in the swarm engine, commented as mock at the call site.
- **Unverified:** whether each of the eleven deployed services actually emits OTEL spans in its own source, and whether the `/metrics` endpoint the ServiceMonitor expects exists on each of them. The ConfigMap sets the exporter target; it does not prove any service calls the SDK.

3.11 Web and product surface readiness

The public-facing surface has its own audit with a composite score of 67 out of 100. It is included because it is what a prospective customer sees before they ever reach the platform.

Table 3.6 — Enterprise readiness scorecard

Dimension	Score	Verdict
Navigation coverage	100%	Fully mapped
Route integrity	99%	Eleven broken internal links, ten placeholder hrefs
Accessibility	92%	WCAG AA, with missing alt text and aria labels outstanding
Performance	100%	Verified
Downloads and assets	100%	Verified
SEO readiness	38%	62% of pages lack generated metadata
Analytics coverage	8%	**Effectively untracked.** Calls to action are not instrumented, so funnel performance is unknowable.
Design system parity	0%	Legacy CSS tokens still present alongside the token system

Analytics coverage at 8% deserves particular attention from anyone evaluating go-to-market claims: it means no assertion about conversion, funnel or campaign performance on the current site can be supported by data. Section 15 treats instrumentation as a prerequisite for the launch plan rather than as a reporting nicety.

3.12 Summary verdict

Verdict	Subsystems
Defend and build on	AI gateway · auth package · forge-api · LangGraph planner · observability stack · network policy · Prisma data model · prompt library · infrastructure-as-code breadth
Wire, do not redesign	Conversation and message persistence · memory and snapshots · the document-chunk-embedding-index chain · policy registration · RBAC enforcement · audit invocation
Replace outright	The TypeScript swarm planner · the simulated worker provider · the AI gateway container entrypoint · the Rust gateway's root proxy route
Decide, then act	HiveSwarm versus platform runtime · Helm generation 1 versus 2 · the eight CI-built, undeployed services · the target service count
Audit before trusting	apps/studio · the three Kotlin services · the C++ ML service · per-service OTEL emission and metrics endpoints

The one-line conclusion. CerebroHive is roughly eighteen months of architecture ahead of its implementation and one critical security finding behind its commercial ambition. The programme in the remainder of this document closes the second gap first, converts a defined slice of the first, and stops adding to either.

4 Programme Design

The programme runs for 24 calendar weeks — 120 working days — organised as six four-week milestones and twelve two-week sprints. Work is divided into seven workstreams that run concurrently with declared interfaces between them, rather than as sequential phases, because the dependencies are genuinely parallel: security cannot wait for the runtime, and the runtime cannot wait for the platform reconciliation.

4.1 Workstreams

Workstream	Charter	Accountable role	Peak FTE
WS0 — Programme & architecture	Decisions D1–D6, architecture decision records, interface contracts between workstreams, weekly risk review	VP Engineering	2
WS1 — Security & identity	Authentication, authorisation, tenant isolation, audit, penetration testing and remediation	Head of Security	2
WS2 — Agent runtime	M10.1 through M10.7: execution, tools, persistence, memory, multi-agent, SDK	Principal Engineer, Runtime	6
WS3 — Platform & infrastructure	Helm reconciliation, secrets, service consolidation, HiveSwarm convergence, tenant isolation at the infrastructure layer	Head of Platform	4
WS4 — Product surfaces	Studio audit and re-platform, CerebroFlow to GA, CerebroSearch delivery	Head of Product Engineering	6
WS5 — Quality & assurance	Test infrastructure, CI gates, evaluation harness, load and chaos testing, SOC 2 evidence automation	Head of Quality	4
WS6 — Go-to-market	Design partners, pricing instrumentation, beta readiness, controlled GA	Head of Commercial	2

4.2 Operating cadence

Ceremony	Frequency	Duration	Output
Sprint planning	Every second Monday	3 hours	Committed sprint backlog with a stated point total
Daily workstream stand-up	Daily	15 minutes	Blockers surfaced within 24 hours
Cross-workstream sync	Weekly, Wednesday	45 minutes	Interface changes and dependency handoffs
Architecture review board	Weekly, Thursday	60 minutes	ADRs accepted or rejected; no ADR, no merge for structural change
Risk review	Weekly, Friday	30 minutes	Risk register updated; contingency release decisions
Sprint review and demo	Every second Friday	90 minutes	Working software demonstrated against the definition of done
Milestone gate	Every fourth Friday	3 hours	Binary pass or fail against the milestone checklist in Section 10
Design partner check-in	Monthly	60 minutes each	Feedback logged against the product backlog

4.3 Governing principles

7. **Finish scaffolds before creating abstractions.** Before adding any package or file, answer four questions: does this capability already exist, is it partially implemented, can it be completed by integration rather than new

code, and would creating it duplicate an existing subsystem. This principle is inherited verbatim from the runtime backlog and it is the single most important constraint in the programme.

8. **No phase starts until the previous definition of done passes against reality.** Not against a mock. Where a phase's assumed reusable component turns out to be less real than documented, that is new information and the plan is updated — the gap is never silently absorbed into the next phase's scope.
9. **Every structural change carries an architecture decision record.** Decisions D1 through D6 and every interface between workstreams are recorded, dated and owned. The record explains what was rejected, not only what was chosen.
10. **Security work cannot be deprioritised by a product deadline.** WS1 has a standing veto on any release that regresses the authentication or isolation posture. This is a structural protection, because in every programme of this kind the pressure to trade it away arrives around week 16.
11. **Estimates are ranges with stated confidence.** Section 7's hour figures carry confidence labels. High-confidence estimates cover wiring work against known schemas; low-confidence estimates cover the Studio audit and the HiveSwarm convergence, where scope is genuinely unknown until discovery completes.
12. **Demonstrate, do not report.** A sprint review shows working software against a real endpoint. A slide describing completed work is not evidence of completed work.

4.4 Definition of done — programme standard

Applies to every work item without exception. Anything that cannot meet it is not done; it is in progress with a plausible narrative attached.

- Code merged to main behind a feature flag where user-visible.
- Unit tests covering the happy path and at least two failure modes.
- An integration test exercising the real dependency, not a mock — for anything touching a provider, database or another service.
- Authentication and authorisation enforced on any new endpoint, with a cross-tenant negative test.
- OTEL span emitted and visible in the trace view; any new failure mode covered by an alert rule or an explicit decision that it is not alert-worthy.
- Cost impact estimated for anything invoking a model, and metered through the Hive Credits path.
- Documentation updated in the same pull request — API reference generated, ADR filed where structural.
- Demonstrated live at sprint review against a deployed environment.

5 Twenty-Four Week Programme Schedule

The Gantt below is the master schedule. Bars are working durations, not elapsed calendar including holidays; milestone markers are gate dates and are fixed. Where a bar starts before its stated dependency completes, the overlap is deliberate and the interface is defined in the corresponding architecture decision record.

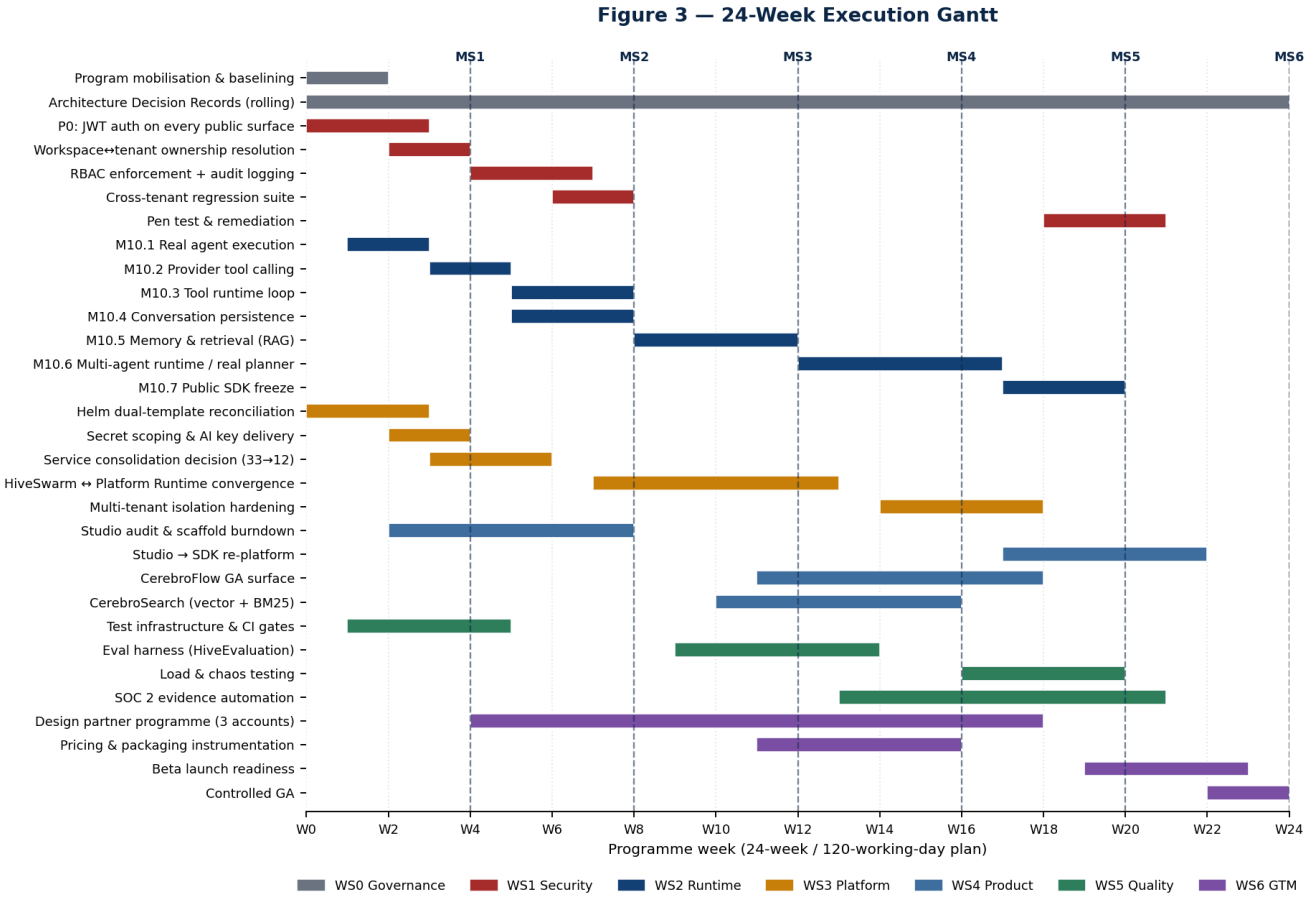

Figure 3 — The 24-week execution Gantt across seven workstreams, with the six milestone gates marked.

5.1 Critical path

The critical path runs through the runtime workstream and is nine tasks long. Any slip on it moves MS6 day for day; slips elsewhere consume float.

Seq	Task	Weeks	Float
1	P0 authentication on every public surface	1–3	Zero — everything else assumes a verified identity
2	M10.1 real agent execution	2–3	Zero
3	M10.2 provider tool calling	4–5	Zero
4	M10.3 tool runtime loop	6–8	Zero
5	M10.5 memory and retrieval	9–12	One week

Seq	Task	Weeks	Float
6	M10.6 multi-agent runtime and real planner	13–17	Zero
7	M10.7 public SDK freeze	18–20	Zero
8	Studio re-platform onto the SDK	18–22	Zero
9	Beta readiness and controlled GA	20–24	Zero

The compression option. If MS6 must move earlier, the only defensible compression is to descope the Studio re-platform to a facade — Studio continues to call internals while the SDK is published for external consumers — which recovers roughly three weeks at the cost of carrying the coupling into horizon two. Compressing the runtime path or the security path is not an option; both have zero float and both are prerequisites for the commercial claim the programme exists to make.

5.2 Dependency map between workstreams

Producer	Consumer	Interface	Due
WS1 security	WS2 runtime	Verified tenant context available on the request object	Week 3
WS1 security	WS3 platform	Token verification requirements for the ingress decision	Week 2
WS3 platform	All	One rendering Helm generation; a stable environment to deploy to	Week 3
WS2 runtime	WS4 product	Conversation API contract, frozen at M10.4	Week 8
WS2 runtime	WS5 quality	Eval hooks on the execution path	Week 10
WS2 runtime	WS4 product	Public SDK surface, frozen at M10.7	Week 20
WS3 platform	WS6 GTM	Metering events attributable to a verified tenant	Week 14
WS5 quality	WS6 GTM	SOC 2 evidence package and penetration test report	Week 22

6 The 120-Day Execution Plan

One hundred and twenty working days, presented week by week. Each week names a theme, its principal deliverables and a single exit signal — the one observable fact that determines whether the week succeeded. Weeks are not interchangeable; the sequence encodes real dependencies.

> **HOW TO USE THIS TABLE**
>
> The exit signal column is the contract. Deliverables can be renegotiated inside a week; the exit signal cannot. If a week ends without its exit signal, that is escalated at the Friday risk review the same day, not absorbed into the following week.

Weeks 1–8 · Foundation and trustworthy runtime

Wk	Theme	Principal deliverables	Exit signal
1	**Mobilise and stop the bleeding**	Cluster verification of D1. Auth hook deployed to staging. Helm render diff produced for both generations. Programme baseline and backlog import. Team onboarding for the first four hires.	A definitive answer on whether the live endpoints require a token.
2	**One deployment truth**	D2 decided; losing Helm generation deleted. Secret scoping reworked so platform-api receives provider keys. M10.1 begins: ai-gateway declared as a dependency, runtime service constructed. Test infrastructure scaffolding.	A single generation renders; render output reviewed and committed as a golden file.
3	**Verified identity in production**	Auth hook in production behind a flag, then enforced. Workspace-to-tenant ownership lookup implemented in the data layer. M10.1 complete. Security regression tests for four access scenarios.	No public endpoint accepts an unauthenticated request; workspace header no longer trusted.
4	**MS1 gate**	M10.2 begins: tool definitions added to the gateway request and response types, Anthropic translation. Service consolidation discovery. Gate review and go/no-go.	MS1 checklist passes in full. D3 and D4 discovery inputs delivered.
5	**Tools, described**	OpenAI tool-call translation. RBAC enforcement begins across platform-api routes. Studio audit begins. Design partner shortlist agreed (D6).	A tool-forcing prompt returns a correctly shaped tool-call array from both providers.
6	**Tools, executed**	M10.2 complete. M10.3 begins: the execution branch replaces always-final-answer; one real tool registered at startup. M10.4 begins in parallel: conversation repository.	The round trip from prompt to tool call to execution to final answer works end to end.
7	**Persistence**	Conversation and message rows written and read. Cross-tenant regression suite lands in CI as a blocking gate. HiveSwarm convergence discovery begins.	Two sequential messages in one conversation show the second is aware of the first.
8	**MS2 gate**	M10.3 and M10.4 complete. Studio audit interim findings. Gate review. Iteration-cap behaviour verified under a looping tool.	MS2 checklist passes. Conversation history survives a process restart.

Weeks 9–16 · Memory, convergence and coordination

Wk	Theme	Principal deliverables	Exit signal
9	**Memory foundations**	M10.5 begins: memory and embedding repositories, embeddings client alongside the chat providers. Evaluation harness design. Convergence options paper for D3.	Embeddings written to the existing pgvector column via the existing schema.
10	**Retrieval**	Retrieval folded into prompt assembly. Eval harness first suite: hallucination, accuracy, safety. CerebroSearch work begins on hybrid ranking.	Seeded facts are retrieved for relevant queries and not for irrelevant ones.
11	**Convergence decision executed**	D3 implemented: one system of record declared, the other retired or formally scoped. CerebroFlow GA scope frozen. Pricing and metering instrumentation begins.	One planner and one execution provider are named as canonical in an accepted ADR.

Wk	Theme	Principal deliverables	Exit signal
12	**MS3 gate**	M10.5 complete. CerebroSearch serving hybrid results. Gate review. First design-partner environment provisioned.	A conversation past the raw context window answers correctly from retrieval.
13	**Real planning**	M10.6 begins: the simulated worker provider is replaced with a real execution provider calling the runtime per node; agent selection wired to the registry.	Swarm nodes produce non-identical, non-canned output.
14	**Plans that differ**	The hard-coded planner is replaced with a model-backed planner emitting a DAG. Metering events flowing and attributable. SOC 2 evidence automation begins.	Two different intents produce two different DAG shapes.
15	**Isolation hardening**	Multi-tenant isolation at the infrastructure layer: per-tenant resource limits, storage partitioning, noisy-neighbour protection. Retry policy implemented in the execution engine.	A tenant cannot exhaust another tenant's execution capacity.
16	**MS4 gate**	M10.6 substantially complete. CerebroFlow feature-complete. Load and chaos testing begins. Gate review.	MS4 checklist passes. Multi-agent orchestration demonstrated on real work.

Weeks 17–24 · Hardening, assurance and launch

Wk	Theme	Principal deliverables	Exit signal
17	**Load and failure**	k6 load profiles against the runtime path. Chaos experiments against the declared degradation map. M10.6 hardening.	Every fallback in the degradation map is observed to occur as specified.
18	**SDK**	M10.7 begins: the facade is built over the frozen internals. Studio re-platform begins. Penetration test kicks off.	The SDK's public surface is documented and stable enough for an external consumer.
19	**Beta preparation**	Beta readiness checklist. Penetration test findings triaged. Studio migrating call sites. Support runbooks drafted.	Every finding has an owner and a target date.
20	**MS5 gate**	M10.7 complete. Penetration remediation cycle one complete. Metering accuracy validated against provider invoices. Gate review.	MS5 checklist passes. Studio runs an agent through the SDK with no internal imports.
21	**Remediation and evidence**	Penetration remediation cycle two. SOC 2 Type I evidence package assembled. Studio re-platform completes. Beta cohort onboarding.	No open critical or high penetration findings.
22	**Beta in anger**	Design partners running production workloads. SLO measurement window opens. Documentation and API reference complete.	Thirty-day SLO measurement window has started with clean instrumentation.
23	**GA readiness**	GA checklist. Pricing enforcement live. Incident response tested by game day. Launch communications prepared.	A simulated Sev-1 is handled within the published response target.
24	**MS6 — controlled general availability**	Controlled GA to the design-partner cohort plus a named waitlist. Retrospective. Horizon-two plan drafted.	MS6 checklist passes. Three accounts in production under contract.

6.1 Buffer and contingency in the schedule

The plan contains three forms of slack, deliberately placed rather than distributed evenly.

- **Gate weeks carry roughly 30% unallocated capacity.** Weeks 4, 8, 12, 16, 20 and 24 are review weeks. Loading them fully guarantees that gate preparation is done at the expense of the gate's honesty.
- **M10.5 carries one week of float.** It is the one runtime phase whose scope is bounded by an existing schema, which makes it the safest place to hold float on an otherwise zero-float path.
- **Sprint 12 is a contingency sprint.** The burndown in Figure 10 projects approximately 34 points of carry-over at 85% of planned velocity. Sprint 12 absorbs it. If velocity holds, sprint 12 pulls horizon-two work forward instead.

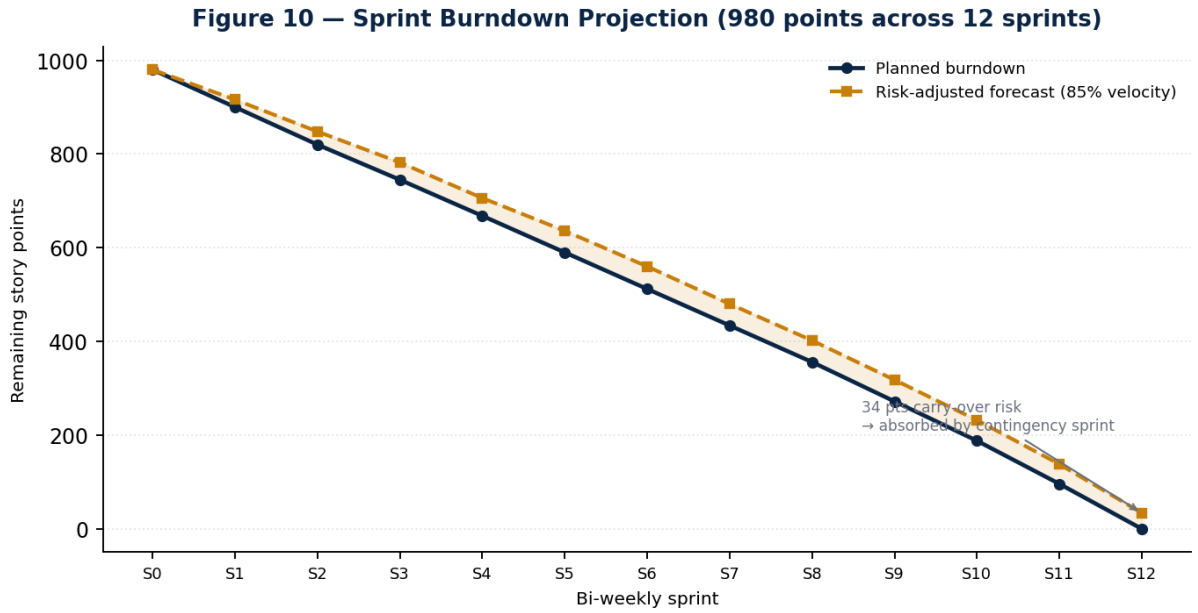

Figure 10 — Sprint burndown projection. The gap between planned and risk-adjusted lines is the case for holding a contingency sprint rather than distributing buffer into every sprint.

7 Effort Model: Hours by Module and Engineer

Total programme effort is estimated at 17,440 engineering hours across 24 weeks. Estimates are bottom-up from the work packages below, then cross-checked top-down against the team ramp in Section 8; the two reconcile to within 4%, which is the closest agreement worth claiming at this stage.

Figure 4 — Effort Distribution (17,440 engineering hours across 24 weeks)

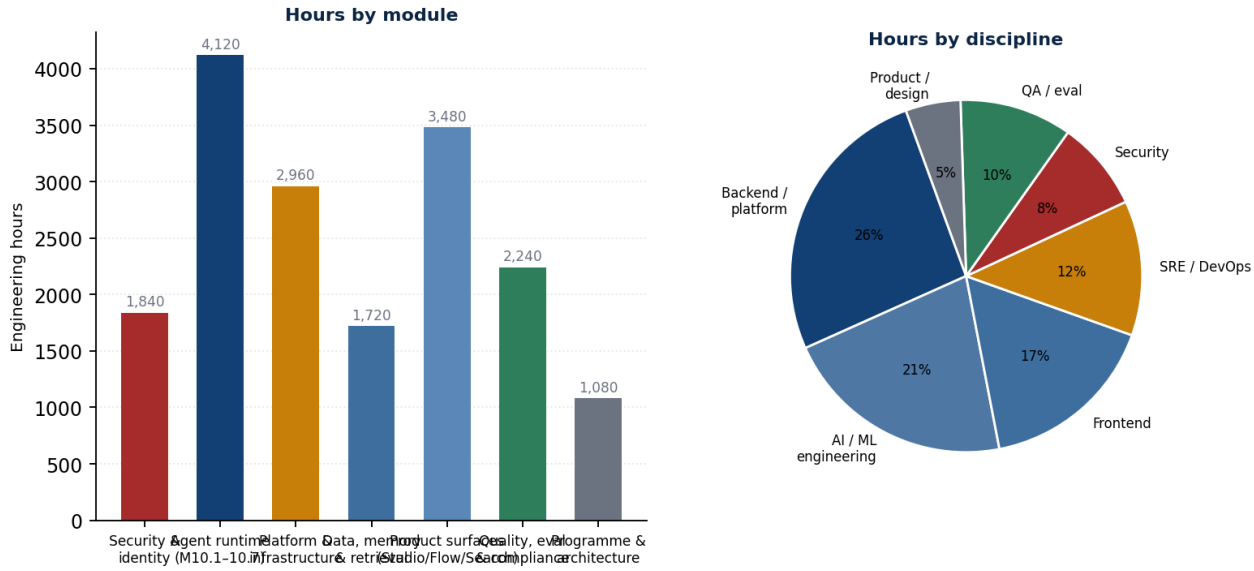

Figure 4 — Effort distribution by module and by discipline.

7.1 Estimation basis and confidence

Confidence	Meaning	Applied to	Planning treatment
High (±15%)	Wiring work against a schema, interface or reference implementation that already exists	M10.1, M10.2, M10.4, auth wiring, Helm reconciliation	Estimate used as stated
Medium (±30%)	New logic, but bounded scope and a known shape	M10.3, M10.5, M10.7, RBAC, metering, CerebroSearch	Estimate used with the upper bound reserved in contingency
Low (±60%)	Scope genuinely unknown until discovery completes	Studio audit and re-platform, HiveSwarm convergence, service consolidation	**Time-boxed discovery first; the estimate is re-baselined at the following gate**

A note on the low-confidence items. Together they account for roughly 4,100 hours — 24% of the programme — on estimates that could be wrong by 60% in either direction. That is the honest number, and it is why weeks 5 through 8 contain time-boxed discovery for all three rather than committed delivery. Anyone reading this plan as a fixed-price commitment should read this paragraph twice.

7.2 Hours by work package

Module / work package	Hours	Roles	Conf.
Security and identity — 1,840 hours			
JWT verification across all public surfaces	280	Backend, security	High
Workspace-to-tenant ownership resolution	180	Backend, security	High
RBAC enforcement across routes	340	Backend, security	Medium
Audit logging tied to verified identity	220	Backend	Medium
Cross-tenant regression suite	260	QA, security	High
Penetration test support and two remediation cycles	420	Security, backend	Medium
Secrets scoping and rotation	140	SRE, security	High
Agent runtime (M10.1–M10.7) — 4,120 hours			
M10.1 real agent execution	220	Backend, AI	High
M10.2 provider tool calling, both providers	380	AI	High
M10.3 tool runtime loop and iteration control	460	AI, backend	Medium
M10.4 conversation persistence	340	Backend	High
M10.5 memory, embeddings and retrieval	780	AI, backend	Medium
M10.6 multi-agent runtime and model-backed planner	1,240	AI, backend	Medium
M10.7 public SDK	420	Backend, AI	Medium
Streaming through the conversation path	280	Backend	Medium
Platform and infrastructure — 2,960 hours			
Helm dual-template reconciliation	320	SRE	High
Service consolidation execution	880	SRE, backend	Low
HiveSwarm and platform runtime convergence	1,020	AI, backend, SRE	Low
Multi-tenant isolation at the infrastructure layer	480	SRE	Medium
Environment and release engineering	260	SRE	High
Data, memory and retrieval — 1,720 hours			
Repository layer for conversations, memory, embeddings	420	Backend	High
Embeddings client and model registry	260	AI	Medium
Hybrid vector and BM25 ranking	540	AI, backend	Medium
Ingestion pipeline and chunking	380	Backend	Medium
Knowledge graph entity extraction, first pass	120	AI	Low
Product surfaces — 3,480 hours			
Studio audit and scaffold burndown	820	Frontend, product	Low
Studio re-platform onto the SDK	1,180	Frontend	Low
CerebroFlow to general availability	960	Frontend, backend	Medium
CerebroSearch product surface	520	Frontend	Medium
Quality, evaluation and compliance — 2,240 hours			
Test infrastructure and CI gates	480	QA	High

Module / work package	Hours	Roles	Conf.
Evaluation harness	620	QA, AI	Medium
Load and chaos testing	440	SRE, QA	Medium
SOC 2 evidence automation	700	Backend, security	Medium
Programme and architecture — 1,080 hours			
Architecture decision records and interface design	440	Principal, architecture	High
Programme management and reporting	400	Programme	High
Design partner technical support	240	Product, backend	Medium
Total	**17,440**		

7.3 Hours per engineer

A full-time equivalent contributes 30 productive engineering hours per week against a 40-hour week — the remaining ten cover ceremonies, review, interviews, incident response and the ordinary friction of working in a large codebase. Planning at 37.5 or 40 is the most common cause of a plan like this failing in week 14 rather than week 24.

Discipline	Avg FTE over 24 wks	Hours available	Hours allocated	Utilisation
Backend / platform	5.3	3,816	3,640	95%
AI / ML engineering	3.5	2,520	2,410	96%
Frontend	3.1	2,232	2,180	98%
SRE / DevOps	2.7	1,944	1,850	95%
Security	1.5	1,080	1,020	94%
QA / evaluation	2.2	1,584	1,510	95%
Product / design	1.0	720	690	96%
Contract and specialist surge	—	4,340	4,140	95%
Total	**19.3 avg**	**18,236**	**17,440**	**96%**

UTILISATION AT 96% IS THE PLAN'S LARGEST HIDDEN RISK

There is almost no absorbency in the model. A single unplanned production incident lasting a week, or one senior departure, consumes the entire margin. This is mitigated by the contingency sprint and the 12% budget contingency, but it should be understood as a live constraint rather than a rounding detail — and it is the specific reason hiring is front-loaded in Section 8.

8 Team Structure and Hiring Plan

The programme starts with 13 full-time equivalents and ends with 26. Thirteen roles are hired across the first fourteen weeks. Hiring is front-loaded deliberately: a hire made in week 16 contributes roughly eight productive weeks after ramp, which is rarely worth the recruiting cost on a 24-week programme.

Figure 5 — Team Composition and Ramp Plan (13 → 26 FTE)

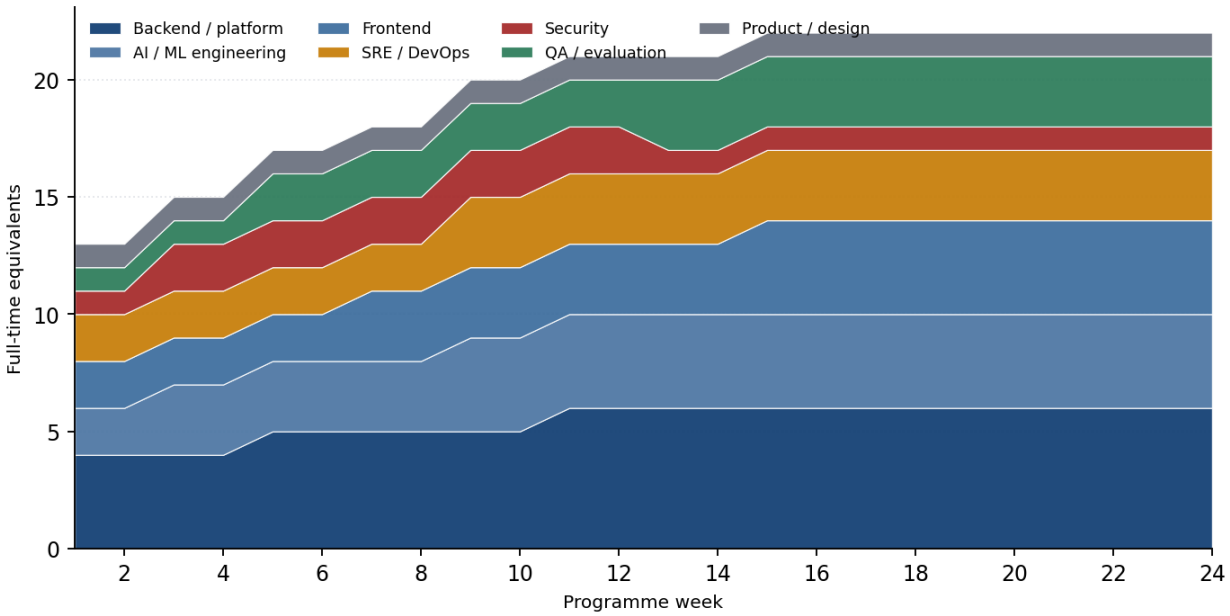

Figure 5 — Team composition and ramp. The step changes correspond to the hiring waves in Section 8.2.

8.1 Target structure at week 24

Function	Roles	FTE	Accountable for
Programme & architecture	VP Engineering, Principal Architect	2	Decisions D1–D6, ADRs, cross-workstream interfaces
Runtime engineering	Principal Engineer, 3 senior AI engineers, 2 backend engineers	6	M10.1 through M10.7, the SDK, evaluation hooks
Platform engineering	Head of Platform, 2 SRE, 1 platform engineer	4	Helm, environments, consolidation, tenant isolation
Product engineering	Head of Product Engineering, 4 frontend, 1 backend	6	Studio, CerebroFlow, CerebroSearch
Security	Head of Security, 1 application security engineer	2	Authentication, authorisation, penetration remediation, SOC 2 controls
Quality	Head of Quality, 2 QA engineers, 1 evaluation engineer	4	Test infrastructure, CI gates, eval harness, load and chaos
Product & design	Senior product manager, product designer	2	Scope, design partner feedback, GA definition
Total		**26**	

8.2 Hiring waves

Wave	Weeks	Roles	Rationale	Risk if missed
Wave 1	0–4	Head of Security, 2 senior AI engineers, 1 SRE	The security and runtime critical paths both start in week 1	**MS1 and MS2 both slip. This wave is non-negotiable.**
Wave 2	4–8	2 backend engineers, 1 QA engineer, 1 frontend engineer	Tool runtime, persistence and the Studio audit all begin	MS3 slips; the Studio audit stays a discovery activity rather than becoming remediation
Wave 3	8–14	1 senior AI engineer, 2 frontend, 1 SRE, 1 evaluation engineer	Multi-agent work, CerebroFlow GA and the eval harness peak together	MS4 slips; eval gates do not exist to protect MS5
Contract / surge	6–22	Penetration testing firm, SOC 2 readiness partner, 1 technical writer	Specialist and time-boxed	Assurance deliverables at MS5 and MS6 have no owner

Recruiting throughput assumption. Nine permanent hires in fourteen weeks in senior AI and platform roles assumes an active pipeline and roughly a six-week average time to offer acceptance. If the pipeline is being built from cold, add four to six weeks and re-baseline MS4 onward. This is assumption A7 in Appendix D and it is the most commonly optimistic number in plans of this shape.

8.3 Ownership map

Nothing in the repository currently declares ownership — there is no CODEOWNERS file and no documented team ownership anywhere the audit could find. That is itself a finding. The following map is established in week 1 and enforced in CI thereafter.

Domain	Owning function	Reviewer of record
packages/auth, all authentication middleware	Security	Head of Security
packages/ai-gateway, providers, routing	Runtime engineering	Principal Engineer, Runtime
packages/capabilities/*, agent runtime service	Runtime engineering	Principal Engineer, Runtime
packages/database, Prisma schema and migrations	Platform engineering	Head of Platform
infra/**, Helm, ArgoCD, Terraform	Platform engineering	Head of Platform
apps/studio, apps/flow, apps/search	Product engineering	Head of Product Engineering
services/agent-runner, planner-service, swarm-*	Runtime engineering	Principal Engineer, Runtime
Kotlin and C++ services	**Unassigned — see D4**	VP Engineering until resolved
tests/**, CI workflows, eval harness	Quality	Head of Quality

8.4 Knowledge risk

Six languages across thirty-three services, none with documented ownership, is a concentration risk in disguise: the knowledge exists but its location is unknown, which behaves identically to a bus factor of one until it is mapped. Three mitigations run from week 1 — pair rotation across the Python, Go and TypeScript agent paths; a mandatory service card for each retained service capturing purpose, dependencies, runbook and owner; and a standing rule that no service survives the consolidation decision without a named owner.

9 Sprint Backlog: Twelve Bi-Weekly Sprints

Nine hundred and eighty story points across twelve sprints, at a planned velocity of roughly 82 points per sprint rising to 92 as the team ramps. Points are relative and calibrated against a reference item — wiring one repository against an existing Prisma model is three points. Acceptance criteria are written to be falsifiable; anything phrased as "works correctly" has been rewritten.

> **BACKLOG DISCIPLINE**
>
> Items may be exchanged within a sprint at equal point value with the workstream lead's agreement. Items may not be added without removing an equivalent. Sprint goals are not negotiable mid-sprint; if the goal becomes unachievable, the sprint is stopped and replanned rather than quietly missed.

9.1 Sprints 1–4 · Foundation

Sprint 1 · Weeks 1–2 · 76 points

Sprint goal: A single deployment truth and a verified identity in staging.

ID	Item	WS	Pts	Acceptance
S1-01	Verify live auth posture on both public hosts (D1)	WS1	5	A written answer with evidence from the cluster, filed as an ADR
S1-02	Render both Helm generations and diff them	WS3	8	Diff committed; conflicting object identities enumerated
S1-03	Decide and delete the losing Helm generation (D2)	WS3	8	One generation remains; chart renders without duplicate object names
S1-04	Deploy the auth preHandler to staging behind a flag	WS1	8	Flag on: unauthenticated requests receive 401. Flag off: prior behaviour
S1-05	Scope secrets so platform-api receives provider keys	WS3	5	Pod environment contains the provider key; gateway reports the provider enabled
S1-06	Declare ai-gateway as a platform-api dependency and construct the runtime	WS2	5	Server boots with the gateway and runtime service constructed
S1-07	Test harness, CI job and coverage baseline	WS5	13	Integration tests run in CI against a real database
S1-08	Programme baseline: backlog import, ownership map, ADR template	WS0	8	CODEOWNERS merged; ADR 0001 through 0003 filed
S1-09	Fix the hard-coded port; read PORT from the environment	WS3	3	Readiness probe passes against the Helm-injected port
S1-10	Studio audit discovery kickoff	WS4	13	Scaffold inventory categorised into remove, complete and keep

Sprint 2 · Weeks 3–4 · 80 points

Sprint goal: MS1: no unauthenticated public endpoint, and one real agent call end to end.

ID	Item	WS	Pts	Acceptance
S2-01	Enforce authentication in production	WS1	8	Every public route returns 401 without a valid token; health checks unaffected
S2-02	Workspace-to-tenant ownership lookup in the data layer	WS1	13	A request naming a workspace the tenant does not own returns 403

ID	Item	WS	Pts	Acceptance
S2-03	M10.1 — replace the hard-coded invoke with a real gateway call	WS2	8	A conversation message returns content from a live provider completion
S2-04	M10.1 negative path — invalid agent id fails cleanly	WS2	3	No silent fallback to a canned response; the error is explicit
S2-05	Four-scenario security regression suite	WS5	13	Authenticated, unauthenticated, invalid token and cross-tenant all covered and blocking
S2-06	Tool definitions added to gateway request and response types	WS2	8	Types compile across all consumers; no provider branching outside the gateway
S2-07	Anthropic tool-call translation	WS2	8	A tool-forcing prompt returns a correctly shaped tool-call array
S2-08	Service consolidation discovery (D4 input)	WS3	13	Every service classified: retain, merge, retire, unknown-owner
S2-09	MS1 gate preparation and review	WS0	6	Checklist evidence assembled ahead of the gate, not during it

Sprint 3 · Weeks 5–6 · 84 points

Sprint goal: Tools described and executed; persistence begins.

ID	Item	WS	Pts	Acceptance
S3-01	OpenAI tool-call translation	WS2	8	Identical response shape to the Anthropic path; regression test for plain prompts
S3-02	M10.3 — execution branch replaces always-final-answer	WS2	13	Tool result is pushed into messages and the loop continues
S3-03	Register one real tool at startup	WS2	5	The registered tool executes against a real dependency
S3-04	Iteration cap behaviour under a looping tool	WS2	5	Cap is enforced and throws cleanly with a diagnosable error
S3-05	Conversation repository following the existing pattern	WS2	8	Create, append and load with history, tested against a real database
S3-06	RBAC enforcement on agent and workflow routes	WS1	13	A role without the permission receives 403; permission map exercised in tests
S3-07	Audit logging tied to the verified identity	WS1	8	Every mutating request writes a record naming the verified subject
S3-08	Studio scaffold burndown wave one	WS4	13	Dead routes removed; placeholder hrefs resolved or ticketed
S3-09	Design partner shortlist and outreach (D6)	WS6	5	Three named candidates with a scheduled first conversation
S3-10	HiveSwarm convergence discovery	WS3	6	Both runtimes' capabilities mapped against one another

Sprint 4 · Weeks 7–8 · 86 points

Sprint goal: MS2: a working agent with tools and memory of its own conversation.

ID	Item	WS	Pts	Acceptance
S4-01	Conversation routes create real rows and load real history	WS2	13	Second message is aware of the first; both rows survive a restart
S4-02	Full round trip test: prompt, tool, execution, final answer	WS5	8	Test runs against the real registered tool in CI
S4-03	Cross-tenant regression suite becomes a blocking CI gate	WS5	8	A pull request that regresses isolation cannot merge
S4-04	Evaluation harness design and first metric definitions	WS5	8	Hallucination, accuracy and safety metrics defined with thresholds

ID	Item	WS	Pts	Acceptance
S4-05	Streaming through the conversation path	WS2	13	Tokens stream to the client; cancellation propagates to the provider
S4-06	Studio audit findings report and remediation plan	WS4	13	Categorised backlog with estimates re-baselined at medium confidence
S4-07	Convergence options paper for D3	WS3	8	Three options with cost, risk and a recommendation
S4-08	Retry policy in the swarm execution engine	WS3	8	Configurable retry with backoff; the code's own TODO removed
S4-09	MS2 gate review	WS0	5	Binary pass or fail recorded with evidence
S4-10	Pricing and metering instrumentation design	WS6	2	Event schema agreed with the runtime team

9.2 Sprints 5–8 · Memory, convergence and coordination

Sprint 5 · Weeks 9–10 · 88 points

Sprint goal: The platform remembers.

ID	Item	WS	Pts	Acceptance
S5-01	Memory and embedding repositories	WS2	13	Rows written to the existing pgvector column via the existing schema
S5-02	Embeddings client alongside the chat providers	WS2	8	Embedding model recorded against each vector
S5-03	Retrieval folded into prompt assembly	WS2	13	Retrieved chunks appear in the assembled prompt and are traceable
S5-04	Retrieval accuracy test with seeded facts	WS5	8	Relevant queries retrieve; irrelevant queries do not
S5-05	Evaluation harness first suite running in CI	WS5	13	The eval gate can block a merge on a regression
S5-06	Hybrid vector and BM25 ranking	WS4	13	Both rankers contribute; the blend is configurable and measured
S5-07	Ingestion pipeline and chunking strategy	WS2	8	A document ingests end to end and becomes searchable
S5-08	CerebroFlow GA scope frozen	WS4	5	Written scope with explicit exclusions
S5-09	Metering events emitted from the runtime	WS6	7	Every model call emits an event attributable to a verified tenant

Sprint 6 · Weeks 11–12 · 90 points

Sprint goal: MS3: convergence executed, memory proven.

ID	Item	WS	Pts	Acceptance
S6-01	D3 decision recorded and communicated	WS0	5	One system of record named in an accepted ADR
S6-02	Convergence execution wave one	WS3	21	The non-canonical path is either retired or formally scoped as a separate product
S6-03	Long-conversation retrieval test beyond the context window	WS5	8	A fact stated far earlier is answered correctly from retrieval
S6-04	Raw-history regression when retrieval finds nothing	WS5	5	The persistence path still works with an empty retrieval result
S6-05	CerebroSearch product surface	WS4	13	Results served with relevance and latency measured
S6-06	SOC 2 evidence automation, control mapping	WS5	13	Events map to named controls; gaps are listed rather than hidden
S6-07	First design partner environment provisioned	WS6	8	Partner can authenticate and run an agent in their own tenant
S6-08	Knowledge graph entity extraction, first pass	WS2	8	Entities extracted and stored; recall measured on a sample

ID	Item	WS	Pts	Acceptance
S6-09	MS3 gate review	WS0	5	Binary pass or fail recorded with evidence
S6-10	Studio scaffold burndown wave two	WS4	4	Scaffold density published as a tracked figure

Sprint 7 · Weeks 13–14 · 92 points

Sprint goal: Plans that actually differ.

ID	Item	WS	Pts	Acceptance
S7-01	Real execution provider replaces the simulated worker	WS2	21	Node output comes from a real model call through a real persona
S7-02	Agent selection wired to the capability registry	WS2	8	Selection varies by node requirement, demonstrably
S7-03	Model-backed planner emitting a DAG	WS2	21	Two different intents produce two different DAG shapes
S7-04	Planner output validation and cycle detection	WS2	8	An invalid or cyclic plan is rejected before execution
S7-05	Metering accuracy reconciliation harness	WS6	8	Metered usage compared against provider-reported usage
S7-06	SOC 2 evidence collection automated for the first five controls	WS5	13	Evidence generated without manual assembly
S7-07	CerebroFlow GA build wave one	WS4	13	Core authoring and execution paths complete behind a flag

Sprint 8 · Weeks 15–16 · 90 points

Sprint goal: MS4: coordinated agents on real work.

ID	Item	WS	Pts	Acceptance
S8-01	End-to-end swarm run with non-identical node outputs	WS5	8	Outputs differ across nodes and are traceable to distinct calls
S8-02	Multi-tenant resource isolation	WS3	21	One tenant cannot exhaust another's execution capacity under load
S8-03	Storage partitioning per tenant	WS3	13	Vector and object storage partitioned and access-checked
S8-04	CerebroFlow feature complete	WS4	21	Every item in the frozen scope is demonstrable
S8-05	k6 load profiles against the runtime path	WS5	13	Published p50, p95 and p99 under the target concurrency
S8-06	Chaos experiments against the degradation map	WS5	8	Each declared fallback observed to occur as specified
S8-07	MS4 gate review	WS0	6	Binary pass or fail recorded with evidence

9.3 Sprints 9–12 · Hardening, assurance and launch

Sprint 9 · Weeks 17–18 · 88 points

Sprint goal: One public surface.

ID	Item	WS	Pts	Acceptance
S9-01	SDK facade over the frozen internals	WS2	21	One call runs any agent with memory, tools and streaming toggled by flags
S9-02	Consumer-side SDK test using only the public surface	WS5	8	No reaching into internals anywhere in the test

ID	Item	WS	Pts	Acceptance
S9-03	Studio re-platform wave one	WS4	21	The highest-traffic paths migrated to SDK calls
S9-04	Penetration test kickoff and environment preparation	WS1	8	Scoped environment with representative data and access
S9-05	Runtime hardening from load findings	WS2	13	Every finding from S8-05 either fixed or explicitly accepted
S9-06	Support runbooks for the eleven retained services	WS3	13	Each runbook exercised once by someone who did not write it
S9-07	Beta readiness checklist drafted	WS6	4	Checklist agreed across all workstreams

Sprint 10 · Weeks 19–20 · 86 points

Sprint goal: MS5: sellable.

ID	Item	WS	Pts	Acceptance
S10-01	SDK frozen; Studio imports no internals	WS2	13	A lint rule fails the build on any internal import from Studio
S10-02	Penetration remediation cycle one	WS1	21	No open critical findings
S10-03	Metering accuracy validated against provider invoices	WS6	13	Within 2% across a full billing period
S10-04	Studio re-platform wave two	WS4	21	Remaining call sites migrated
S10-05	Documentation and generated API reference complete	WS0	8	Reference generated from source in CI, not maintained by hand
S10-06	MS5 gate review	WS0	5	Binary pass or fail recorded with evidence
S10-07	Pricing enforcement behind a flag	WS6	5	Credit exhaustion produces the specified behaviour, not an error

Sprint 11 · Weeks 21–22 · 84 points

Sprint goal: Evidence and partners in production.

ID	Item	WS	Pts	Acceptance
S11-01	Penetration remediation cycle two	WS1	13	No open critical or high findings
S11-02	SOC 2 Type I evidence package assembled	WS5	21	Auditor-ready package with gaps explicitly listed
S11-03	Design partners running production workloads	WS6	13	Three tenants with real usage and metered consumption
S11-04	SLO definitions published and measurement window opened	WS5	13	Availability, latency and quality SLOs with alerting attached
S11-05	Studio re-platform complete and scaffold figure published	WS4	13	Tracked figure with a trend line, not a point measurement
S11-06	Incident response game day	WS3	8	A simulated Sev-1 handled within the published response target
S11-07	Beta cohort onboarding automation	WS6	3	A new tenant is provisioned without manual steps

Sprint 12 · Weeks 23–24 · 36 points

Sprint goal: MS6: controlled general availability, plus contingency absorption.

ID	Item	WS	Pts	Acceptance
S12-01	GA checklist completion	WS0	8	Every item evidenced; exceptions signed off by a named owner

ID	Item	WS	Pts	Acceptance
S12-02	Launch communications and enablement	WS6	5	Sales and support enabled against the actual capability, not the roadmap
S12-03	Controlled GA to the design-partner cohort and waitlist	WS6	8	Contracts in place; usage flowing; billing accurate
S12-04	Programme retrospective and horizon-two baseline	WS0	5	Written retrospective; the next plan drafted from measured velocity
S12-05	Contingency absorption	All	10	Approximately 34 projected carry-over points cleared

9.4 Velocity assumptions

Sprint range	Planned velocity	Rationale
1–2	76–80	Team of 13 to 17; onboarding drag from wave-one hires
3–6	84–90	Wave two productive; codebase familiarity improving
7–11	84–92	Full team; offset by increasing coordination cost
12	36	Contingency sprint, deliberately light
Total	**980 points**	Risk-adjusted forecast at 85% leaves roughly 34 points, absorbed by sprint 12

10 Milestone Checklists

Six gates, four weeks apart. Each is binary. A gate that passes with three exceptions has not passed; it has been renegotiated, and the renegotiation is recorded as a change to the plan rather than as a milestone achievement. The evidence column exists so that a gate cannot be passed on assertion.

MS1 — Trustworthy front door (end of week 4)

The single most important gate in the programme. Everything downstream assumes a verified identity and a single deployment truth; if either is unresolved here, the remaining twenty weeks are built on the same sand.

✓	Criterion	Evidence required	Owner
☐	No public endpoint accepts an unauthenticated request	Automated test hitting every route without a token; all return 401	Head of Security
☐	Tenant identity is sourced from verified token claims only	Code search confirming no route reads x-tenant-id or x-user-id	Head of Security
☐	Health checks remain unauthenticated and pods reach Ready	Kubernetes rollout with readiness passing	Head of Platform
☐	Exactly one Helm template generation renders	Committed render output; no duplicate object identities	Head of Platform
☐	platform-api receives provider API keys in its environment	Pod environment inspection; gateway reports the provider enabled	Head of Platform
☐	A conversation message returns real provider content	Response shows a real model id and duration; revoking the key produces a real failure, not a canned string	Principal Engineer, Runtime
☐	An invalid agent id fails explicitly	Test asserting an error rather than a silent fallback	Principal Engineer, Runtime
☐	D1 answered and D2 decided, both recorded as ADRs	Merged ADRs with dates and owners	VP Engineering

Gate rule: If the authentication criteria fail, the programme stops and all workstreams redirect to WS1 until they pass. No other criterion may be traded against them.

MS2 — Working agent (end of week 8)

The agent becomes useful rather than merely responsive: it can use a tool and it can remember what was said to it.

✓	Criterion	Evidence required	Owner
☐	A tool-forcing prompt returns a correctly shaped tool-call array from both providers	Integration test, one per provider	Principal Engineer, Runtime
☐	A normal prompt still returns plain content with no tool calls	Regression test proving MS1's path is intact	Principal Engineer, Runtime
☐	The full round trip completes: prompt, tool call, real execution, final answer	End-to-end test through the HTTP route	Principal Engineer, Runtime
☐	The iteration cap throws cleanly on a looping tool	Test with a deliberately non-resolving tool	Head of Quality
☐	Two sequential messages show the second is aware of the first	Conversation continuity test	Principal Engineer, Runtime
☐	Message rows survive a process restart	Direct database query after restart	Head of Platform
☐	RBAC is enforced, not merely available	A role lacking the permission receives 403 in test	Head of Security

✓	Criterion	Evidence required	Owner
☐	Cross-tenant access is impossible and proven by a blocking CI gate	A deliberately regressing pull request fails to merge	Head of Quality
☐	Every mutating request writes an audit record naming the verified subject	Audit query across a test run	Head of Security

Gate rule: Persistence and tool execution are both required. Passing on tools alone defers the harder half into a sprint that has no room for it.

MS3 — Remembering agent and one runtime (end of week 12)

Memory becomes real, and the organisation stops paying to build the same capability twice.

✓	Criterion	Evidence required	Owner
☐	A conversation beyond the raw context window answers correctly from retrieval	Long-conversation test with a fact stated early	Principal Engineer, Runtime
☐	Seeded facts retrieve for relevant queries and not for irrelevant ones	Retrieval accuracy test with a published precision figure	Head of Quality
☐	The raw-history path still works when retrieval returns nothing	Regression test	Head of Quality
☐	D3 is decided, recorded and being executed	Merged ADR plus visible convergence work in the sprint	VP Engineering
☐	CerebroSearch serves hybrid vector and keyword results	Live demonstration with latency and relevance measured	Head of Product Engineering
☐	Metering events are emitted and attributable to a verified tenant	Event stream sampled and reconciled	Head of Commercial
☐	The evaluation gate can block a merge on a quality regression	A deliberately regressing pull request fails the gate	Head of Quality
☐	One design partner can authenticate and run an agent in their own tenant	Partner-witnessed session	Head of Commercial

Gate rule: D3 must be executed, not merely decided. A recorded decision with no visible convergence work does not pass this gate.

MS4 — Coordinating agents (end of week 16)

The multi-agent claim becomes defensible. This is the gate at which the level-four maturity positioning becomes honest.

✓	Criterion	Evidence required	Owner
☐	Two different intents produce two different DAG shapes	Test proving the planner is not returning a fixture	Principal Engineer, Runtime
☐	Each node's output comes from a real model call through a real persona	Trace showing distinct calls with distinct personas	Principal Engineer, Runtime
☐	An invalid or cyclic plan is rejected before execution	Test with a deliberately cyclic plan	Principal Engineer, Runtime
☐	Retry policy is implemented in the execution engine	Configurable retry with backoff, exercised under induced failure	Head of Platform
☐	One tenant cannot exhaust another's execution capacity	Load test with an adversarial tenant profile	Head of Platform

✓	Criterion	Evidence required	Owner
☐	CerebroFlow is feature-complete against its frozen scope	Every scope item demonstrated live	Head of Product Engineering
☐	p50, p95 and p99 published under target concurrency	k6 report committed	Head of Quality
☐	Every declared fallback in the degradation map is observed	Chaos experiment report, one row per fallback	Head of Quality

Gate rule: The planner criterion is the gate. A multi-agent system running a fixed plan is a demonstration, not a product.

MS5 — Sellable platform (end of week 20)

The platform acquires a stable public surface, a clean security report and accurate billing — the three things a commercial conversation cannot proceed without.

✓	Criterion	Evidence required	Owner
☐	One SDK call runs any agent with memory, tools and streaming toggled by flags	Consumer-side test using only the public surface	Principal Engineer, Runtime
☐	Studio imports no internal packages	A lint rule fails the build on any internal import	Head of Product Engineering
☐	No open critical penetration findings	Tester's report with remediation evidence	Head of Security
☐	Metered usage is within 2% of provider-reported usage	Reconciliation over a full billing period	Head of Commercial
☐	Credit exhaustion produces the specified behaviour	Test against an exhausted tenant	Head of Commercial
☐	API reference is generated from source in CI	Build artefact, not a hand-maintained document	VP Engineering
☐	Runbooks exist for every retained service and have been exercised	Each runbook signed off by someone who did not author it	Head of Platform

Gate rule: The 2% metering tolerance is deliberate. Billing a customer for consumption you cannot reconcile is a commercial risk that outweighs the schedule cost of fixing it.

MS6 — Controlled general availability (end of week 24)

The programme's terminal gate. Three accounts in production under contract, with the evidence to survive a security review.

✓	Criterion	Evidence required	Owner
☐	Three design-partner accounts in production under contract	Signed agreements plus live usage	Head of Commercial
☐	No open critical or high penetration findings	Second remediation cycle evidence	Head of Security
☐	SOC 2 Type I evidence package assembled with gaps listed	Auditor-ready package	Head of Quality
☐	Published SLOs with 30 days of measured attainment	Dashboard with the full window populated	Head of Quality
☐	A simulated Sev-1 handled within the published response target	Game day report	Head of Platform
☐	Billing accurate and enforced for all three tenants	Invoice reconciliation	Head of Commercial
☐	A new tenant provisions without manual steps	Timed provisioning run	Head of Platform
☐	Retrospective complete and horizon two baselined on measured velocity	Written retrospective and a baselined plan	VP Engineering

Gate rule: Controlled means controlled. General availability to an open funnel requires self-serve signup and support scaling, neither of which is in this programme's scope.

11 Risk Register and Mitigation Plan

Twelve risks are tracked at programme level. Each has a named owner, a mitigation that is funded in the plan, and a trigger — the observable event that converts the risk into an issue and releases contingency. Risks without triggers are wishes; the trigger column is the part that makes this register operational.

Figure 7 — Risk Heat Map

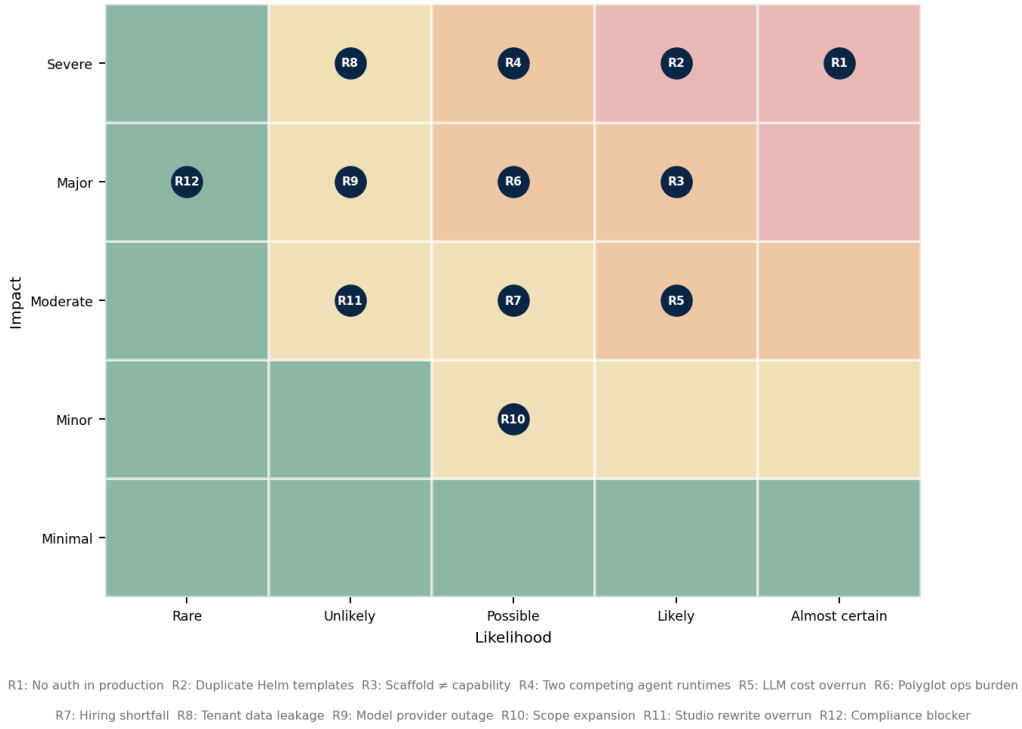

Figure 7 — Risk heat map. Position reflects the assessment before mitigation; the register below states the residual position after.

11.1 Register

ID	Risk	Owner	Mitigation (funded)	Trigger	Residual
R1	Production APIs are currently unauthenticated and may already have been accessed	Head of Security	D1 cluster verification in week 1; auth enforced by week 3; audit log review for anomalous cross-tenant access patterns; incident response plan pre-drafted	D1 confirms no token is required today	**High until week 3, then Low**
R2	Two Helm generations produce unpredictable production state	Head of Platform	Render diff in week 1; one generation deleted in week 2; golden render file committed and diffed in CI thereafter	Any deployment produces an object the chart did not intend	Low after week 2
R3	Scaffolded packages are mistaken for capability in planning or in sales	VP Engineering	Section 3 published internally and to design partners; scaffold density tracked and reported; the constitution's product list annotated with implementation status	Any external commitment references a product not in the programme scope	Medium — this risk is cultural and does not fully retire
R4	Two agent runtimes diverge further before D3 is executed	VP Engineering	D3 dated week 4, executed by week 12; a freeze on new capability in the non-canonical path from week 4	Either runtime gains a capability the other lacks after week 4	Medium until week 12, then Low
R5	LLM inference cost exceeds the $468k budget line	Head of Commercial	Per-tenant and per-environment budget caps enforced in the gateway; the existing token-budget alert wired to a hard stop; weekly cost review from week 2	Weekly spend exceeds 1.3× the run rate implied by the budget	Medium

ID	Risk	Owner	Mitigation (funded)	Trigger	Residual
R6	Six-language operational burden exceeds the SRE team's capacity	Head of Platform	D4 consolidation decision in week 6; service cards mandatory; no service retained without a named owner	Mean time to restore exceeds target on any service twice in a month	Medium
R7	Hiring shortfall against the nine-hire plan	VP Engineering	Wave-one hiring started before week 0; contract surge capacity budgeted; scope compression option pre-identified in Section 5.1	Any wave is more than two weeks behind at its close	Medium
R8	Cross-tenant data leakage occurs before isolation is proven	Head of Security	Cross-tenant regression suite as a blocking CI gate from week 8; storage partitioning by week 16; penetration test from week 18	Any test or report shows data crossing a tenant boundary	**High until week 8, then Low**
R9	Model provider outage or material capability change	Principal Engineer, Runtime	The gateway's existing failover across providers; eval harness re-run on any provider or model version change; no provider-specific logic outside the gateway	Any provider incident exceeding one hour, or a deprecation notice	Low — this is the mitigation the gateway was built for
R10	Scope expansion, particularly requests to advance additional products	VP Engineering	Section 1.5 published as the explicit exclusion list; any addition requires an equivalent removal at the milestone gate	Any work item enters a sprint without a corresponding removal	Medium
R11	Studio re-platform overruns its low-confidence estimate	Head of Product Engineering	Time-boxed discovery in weeks 3–8 with a re-baseline at MS2; the facade compression option in Section 5.1 held in reserve	Re-baselined estimate exceeds 1,600 hours at MS2	Medium
R12	A compliance finding blocks the enterprise sales motion	Head of Quality	SOC 2 evidence automation from week 14; gaps listed rather than concealed; readiness partner engaged from week 6	The readiness partner identifies a control with no feasible path by MS5	Low

11.2 Contingency release policy

The 12% budget contingency and the sprint-12 schedule contingency are held centrally and released only against a triggered risk, by the VP Engineering, with the release recorded in the risk register. This exists to prevent the ordinary failure mode in which contingency is consumed early by optimism and unavailable late when it is needed.

Trigger class	Release authority	Maximum release
A security risk (R1, R8) triggers	Immediate, no approval required	Unlimited within the contingency line
A critical-path risk (R4, R7, R11) triggers	VP Engineering	25% of the remaining contingency per event
A cost risk (R5) triggers	VP Engineering with Head of Commercial	Requires a corresponding scope or usage reduction
Any other risk triggers	Milestone gate decision	10% of the remaining contingency per event

11.3 Risks accepted without mitigation

Two risks are consciously accepted. Naming them is more useful than pretending they are covered.

- **Frontier model pricing changes materially.** The commercial model passes inference cost through as Hive Credits, which transfers most of the exposure to the customer, but a step change in pricing would alter the unit economics of every edition. No mitigation is funded; the response is repricing, which is a commercial decision rather than an engineering one.
- **A competitor ships an equivalent governed multi-agent platform inside the window.** The programme's response would be to compete on the knowledge graph and governance depth identified as the durable

moats in Section 2.4 — neither of which is accelerable inside 24 weeks. This is accepted rather than mitigated, and it is the strongest argument for not extending the timeline further.

12 Budget and Infrastructure Estimates

Total programme cost is estimated at $5,825,000 over 24 weeks. Figures are planning estimates in US dollars, exclusive of tax, based on the assumptions in Appendix D. Payroll dominates at 67%, which is normal and correct for a programme whose deliverable is engineering capability rather than infrastructure.

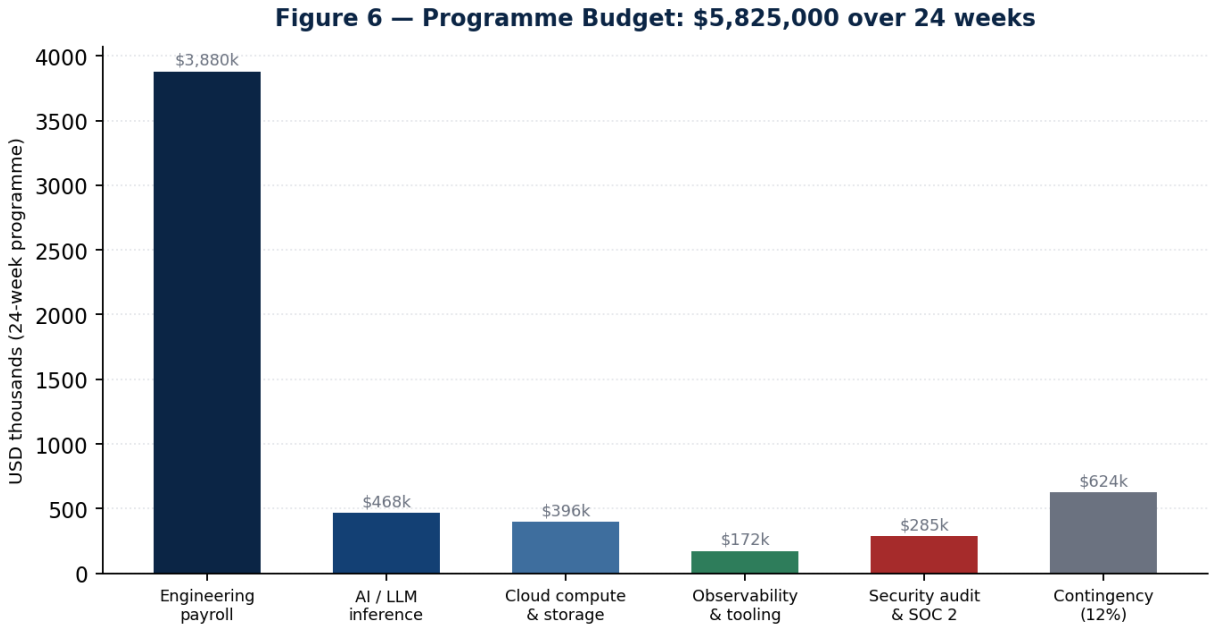

Figure 6 — Programme budget by category.

12.1 Budget detail

Table 12.1 — Twenty-four week budget

Category	Basis	24-week cost	Run-rate after
Engineering payroll			
Permanent engineering	19.3 average FTE, blended $198/hr fully loaded	$3,270,000	$710,000/month at 26 FTE
Contract and specialist surge	Technical writer, specialist contractors	$310,000	Tapers to zero
Programme, product and design	3 FTE across the period	$300,000	$65,000/month
AI and inference			
Development and testing inference	Estimated 340M tokens across the period, blended rate	$182,000	$34,000/month
Evaluation harness runs	Nightly suites from week 10; regression runs per merge	$96,000	$21,000/month
Design partner workloads	Three tenants from week 12, absorbed pre-billing	$118,000	Passed through as credits
Embeddings and indexing	Initial corpus ingestion plus incremental	$72,000	$11,000/month
Cloud infrastructure			

Category	Basis	24-week cost	Run-rate after
Compute — three environments	Production, staging, development; Kubernetes node pools	$186,000	$36,000/month
GPU allocation	Embeddings and the C++ ML service	$84,000	$16,000/month
Storage — object, vector, database	pgvector, object store, backups via Velero	$78,000	$15,000/month
Network and egress	Ingress, cross-zone, provider egress	$48,000	$9,000/month
Observability and tooling			
Metrics, logs and traces	Prometheus, Thanos, Loki, OTEL collector footprint	$92,000	$18,000/month
CI and build	GitHub Actions minutes, Turborepo remote cache, registries	$46,000	$9,000/month
Licences and SaaS	Design, project, security and productivity tooling	$34,000	$7,000/month
Security and compliance			
External penetration test	Two cycles: initial and post-remediation retest	$145,000	Annual
SOC 2 Type I readiness partner	Engaged from week 6 through MS6	$110,000	Type II is a subsequent engagement
Secrets, scanning and supply chain	Gitleaks, Semgrep, CodeQL, dependency scanning	$30,000	$6,000/month
Contingency (12%)	Held centrally, released per Section 11.2	**$624,000**	—
Total		**$5,825,000**	**~$957,000/month steady state**

12.2 Infrastructure sizing

Sizing assumes three environments with production carrying roughly 70% of the footprint, and reflects the consolidated service estate assumed by decision D4 rather than the current thirty-three services.

Environment	Nodes	Databases	Notes
Production	12–18 nodes with autoscaling via KEDA	Postgres with pgvector, primary plus read replica; Redis; NATS	Multi-AZ; Velero backups; Thanos long-term metric retention
Staging	6–8 nodes	Postgres primary; Redis	Production-shaped for the penetration test and load runs
Development	4–6 nodes	Shared Postgres; Redis	Ephemeral namespaces per pull request from week 4
Design partner tenants	Isolated namespaces within production	Partitioned storage per tenant from week 16	Provisioned automatically from week 22

12.3 Cost controls

- **Per-tenant and per-environment inference budgets enforced in the gateway.** The gateway already tracks cost per request; the control is to make the existing token-budget alert a hard stop rather than a notification.
- **Response caching enabled in non-production.** The cache already exists in the gateway. Development and evaluation traffic is highly repetitive and is the cheapest place to recover inference spend.
- **Ephemeral pull-request environments with automatic teardown.** Idle non-production infrastructure is the most common source of unplanned cloud spend in programmes of this shape.

- **Weekly cost review from week 2, owned by the Head of Commercial.** Reviewed against run rate, not against total budget, so that overrun is visible in week 6 rather than week 20.
- **Model routing by task class.** Planning and reasoning use frontier models; classification, extraction and routing use smaller models. This is a gateway configuration, not new engineering, and it is typically worth 30–40% of inference spend.

12.4 What this budget does not include

Sales and marketing headcount; customer success; general and administrative costs; office and equipment; legal and contracting; and any horizon-two engineering. It is an engineering programme budget, not a company operating budget, and it should not be read as one.

13 Target Technical Architecture

This section describes the week-24 state, not the ten-year vision. Everything below is either built during the programme or already exists and is retained. Where the target diverges from the five-tier model in the constitution, the divergence is stated and justified rather than quietly absorbed.

13.1 The request path

One path, enforced, with identity established at the edge and carried unbroken through planning, execution, memory and audit. The most important property is that there is exactly one of it — today there are effectively two, and neither is fully enforced.

Figure 9 — Target Request Path: intent → plan → act → audit

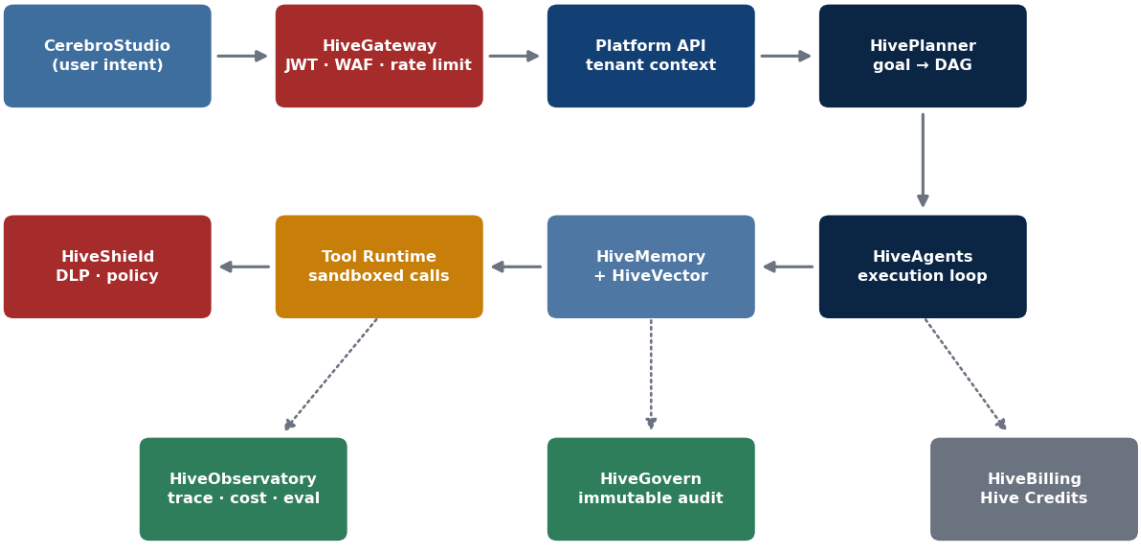

Figure 9 — Target request path. Solid arrows are the synchronous path; dotted arrows are asynchronous emission of telemetry, audit and metering.

13. **Intent arrives at the gateway.** TLS terminates, the WAF and OWASP core rule set apply, rate limiting applies per organisation, and the bearer token is verified against Keycloak JWKS. An unverified request goes no further. This is the control that does not exist today.
14. **Tenant context is established.** Organisation, subject, roles and permissions come from verified token claims. Workspace is resolved through an ownership lookup in the data layer, not from a header. Every downstream check now operates over a trustworthy input.
15. **The planner decomposes the goal.** A model-backed planner emits a directed acyclic graph, validated for cycles before execution. One planner, canonical per decision D3.
16. **The runtime executes the graph.** Nodes dispatch through the existing execution engine — topological ordering, priority queue, worker-pool capacity, cancellation tokens, failure cascade — to a real execution provider calling the runtime service per node. Agent selection uses the existing capability registry.

17. **Memory participates on both sides.** Relevant chunks are retrieved and folded into prompt assembly before the call; results are written back as messages and, where durable, as memory.
18. **Tools execute under policy.** The tool runtime invokes registered tools; the policy engine — with policies actually registered — and DLP evaluate the call before it leaves the boundary.
19. **Everything is observed, audited and metered.** An OTEL span per hop, an immutable audit record per action keyed on the verified subject, and a metering event per model call attributable to a real tenant. The third of these is what makes the commercial model executable.

13.2 Service estate at week 24

Decision D4 targets a consolidation from thirty-three services to a defensible estate. The recommendation below is the input to that decision, not its outcome.

Retained	Language	Rationale
Gateway (edge)	Rust or ingress-native	Single enforced entry point. Retain the Rust gateway only if the root-proxy defect is fixed and it is actually placed in the ingress path; otherwise consolidate into ingress plus in-service verification
platform-api	TypeScript	Conversation, agent and workflow API. The system of record for the single-agent path
forge-api	TypeScript	Developer environment backend. Already the reference implementation for authentication
agent-runner	Python	Multi-agent orchestration. Retained if D3 selects HiveSwarm as canonical for orchestration
planner-service	Python	The best planning implementation in the estate. Retained under either D3 outcome
swarm-api	Go	HTTP and WebSocket surface with NATS and Redis for the orchestration path
tool-gateway	Go	Tool execution boundary. Extended to serve both agent paths rather than only HiveSwarm
memory-service	Python	Retained subject to a source read; merged into the runtime if it proves thin
temporal-worker	TypeScript	Durable workflow execution for CerebroFlow
ml-svc	C++	Embeddings and scoring. Retained on performance grounds; requires an owner
Studio, Forge and Flow front ends	TypeScript	Product surfaces, all consuming the SDK

Candidates for retirement or merge: the TypeScript swarm-runtime once its scheduler is either adopted or superseded; router-service if selection consolidates into the registry; evaluation-service and learning-service if their capability lands in the eval harness; the three Kotlin services pending an ownership and usage decision; and the sixteen services that contain no source at all, which should be deleted rather than carried.

13.3 Data architecture

The data model does not change. It is already correct, already migrated and already live — the programme populates it rather than redesigning it, and this is the single largest source of schedule confidence in the plan.

Concern	Model	Programme action
Conversation state	AgentConversation, AgentMessage with memory and artifacts as JSON, role, content, tool invocations and metadata	Build repositories; wire routes. No migration

Concern	Model	Programme action
Agent memory	Memory, MemorySnapshot	Build repositories; implement summarisation and retrieval in the reserved capability package
Retrieval corpus	Document → DocumentVersion → Chunk → Embedding (pgvector 1536) → VectorIndex	Build the ingestion pipeline and embeddings client. No migration
Execution history	AgentExecution	Populate from the runtime; source for evaluation and cost attribution
Tool catalogue	Tool, ToolVersion, ToolCategory, AgentTool with an existing repository	Register real tools; no schema work
Policy	Policy model with an instantiated engine	Register real policies. The model and engine already exist
Tenancy	Organisation and workspace	**Add the ownership lookup that closes the workspace header gap**

13.4 Non-functional targets at week 24

Property	Target	How it is measured
Availability	99.5% for the conversation API, measured monthly	Prometheus service-down and error-rate rules already defined
Latency	p95 under 2.5s to first token; p99 under 8s for a single-tool completion	Existing p99 latency alert plus k6 profiles
Throughput	200 concurrent agent executions per tenant without cross-tenant degradation	Load test with an adversarial tenant profile
Cold start	Full platform boot under 45 minutes per the declared sequence	Timed cold-start run against a clean namespace
Degradation	Every fallback in the map occurs as declared	Chaos experiment per fallback
Recovery	MTTR under 60 minutes for Sev-1	Game day plus incident record
Cost attribution	Metered usage within 2% of provider-reported usage	Reconciliation over a full billing period
Isolation	Zero cross-tenant access in test or in production audit	Blocking CI gate plus audit log review

13.5 Declared divergences from the constitution

Three, each deliberate.

- **Tier 2's HiveGateway may be delivered by ingress plus in-service verification rather than by a distinct Rust service.** The constitution names HiveGateway as a tier-2 product. The implementation question — dedicated service or ingress-plus-library — is an engineering choice and should not be forced by the diagram. The property that matters is that verification is unavoidable, not where the code lives.
- **Sixty-plus tier-3 and tier-4 products remain unimplemented and are not represented in the week-24 architecture.** The capability matrix should be annotated with implementation status so that the architecture document stops reading as a description of what exists.
- **HiveExchange, HiveBilling and HiveLicense at tier 5 are represented only by metering events.** Full commerce infrastructure follows this programme. The metering event schema is designed to be their input, so this is a deferral rather than a divergence in direction.

14 Product Roadmap Beyond the Programme

The constitution's eight-phase roadmap is a decade-scale artefact and remains the strategic frame. What follows locates this programme within it and states what the two horizons after it actually require, in the same evidence-based register as the rest of this document.

Horizon	Window	Objective	Principal prerequisites
H1 — Trustworthy core	This programme, weeks 1–24	Verified identity, real agent runtime, memory, one orchestration path, three products at GA	The six decisions in Section 1.4
H2 — Enterprise platform	Months 7–12	Governance depth, SOC 2 Type II, Enterprise edition deployability, CerebroInsight and CerebroArchive to GA	Private-cloud deployment model; a self-serve provisioning path; support scaling
H3 — Intelligence mesh	Months 12–24	Knowledge graph at enterprise scale, cross-product memory, CerebroERP and CerebroCRM to beta	The knowledge graph moat requires sustained investment starting in H2, not H3
H4 — Marketplace	Year 2–3	HiveExchange with partner-authored assets and a 20% platform fee	A stable SDK at liquidity, billing infrastructure, a partner programme, and enough tenants to make listing worthwhile
H5 — Developer platform and verticals	Year 3–5	HiveForge and HiveAPI public; vertical SKUs	Public API stability guarantees; industry compliance programmes per vertical

THE SEQUENCING RISK THAT MATTERS MOST

The knowledge graph is identified in Section 2.4 as the most durable moat in the portfolio and is currently the least built. It is scheduled in H3 by the constitution's phasing. If it is genuinely the moat, investment should begin in H2 — a graph accumulates value only through time in production, and starting it a year later delays the defensibility by a year with no way to compress it later.

15 Launch and Go-to-Market Plan

The launch plan is deliberately narrow, for the same reason the engineering plan is: a controlled launch that three customers succeed with is worth more than an open launch that fifty evaluate and abandon. The plan runs in parallel with engineering from week 5 and is gated by the same milestones.

15.1 Sequence

Phase	Weeks	Objective	Gate
Design partner selection	5–8	Three named accounts under a design-partner agreement with defined success criteria and feedback obligations	D6 decided by week 5
Guided onboarding	12–16	Partners authenticate and run agents in their own tenants with hands-on support	MS3 passed
Production workloads	16–22	Partners run real work; usage is metered; feedback drives the backlog	MS4 passed
Beta readiness	19–22	Documentation, runbooks, support paths and pricing enforcement complete	MS5 passed
Controlled GA	23–24	Contracts signed, billing accurate, waitlist opened	MS6 passed

15.2 Design partner criteria

Three partners, selected against criteria rather than availability. The wrong partner costs more than no partner, because their feedback distorts the backlog for the remainder of the programme.

- A mid-market or enterprise-department buyer matching the Business edition profile, since that is the edition this programme makes deliverable.
- A real, bounded, recurring workflow with a measurable baseline — not an exploratory pilot. Without a baseline there is no ROI story at MS6.
- A technical sponsor willing to run a security review, because surviving one before GA is more valuable than a favourable reference.
- Tolerance for a controlled release, expressed contractually, including a defined support model and explicit limitations.
- Diversity of workload shape across the three: one document-and-retrieval heavy, one automation heavy, one analysis heavy. Three similar partners produce one data point.

15.3 Positioning

The positioning must track the maturity model rather than lead it. At MS6 the platform credibly delivers levels one through four — assistant, copilot, autonomous agent and multi-agent team. It does not deliver level five department automation or level six intelligence mesh. Selling those is the fastest available way to lose a reference account permanently, and reference accounts are the entire asset this programme is building.

Audience	Message	Proof point available at MS6
Enterprise buyer	Governed multi-agent automation with verified identity, per-action audit and attributable cost	Penetration test report, SOC 2 Type I evidence package, published SLOs with 30 days of attainment

Audience	Message	Proof point available at MS6
Technical evaluator	One API, one identity model, real tool execution, retrieval-backed memory, open deployment	Live SDK, generated API reference, degradation map validated by chaos testing
Economic buyer	Consumption pricing that does not penalise automation; consumer seats free	Metering reconciled to within 2% of provider invoices
Partner or ISV	Embeddable platform via a frozen SDK	SDK stability commitment and a versioning policy
Investor	A platform whose principal technical risks are retired and whose commercial model is now executable	Six passed gates with published evidence; three accounts in production

15.4 Prerequisites the launch plan depends on

Three are outside engineering and are frequently forgotten until they are urgent.

- **Analytics instrumentation.** Coverage is at 8%. Until calls to action are tracked, no funnel or campaign claim can be supported by data. This must be fixed before, not during, the launch phase.
- **SEO and metadata.** 62% of pages lack generated metadata. Organic discovery is effectively disabled, which matters most for the waitlist opened at MS6.
- **Support model.** Business edition commits to 24/7 support with a four-hour response. Meeting that commitment for three tenants requires a defined rota and runbooks, both of which are in scope, and a staffing decision, which is not.

15.5 Success measures for the launch

Measure	Target at week 24	Why this measure
Design partners in production under contract	3 of 3	The binary test of whether the platform is usable by someone who did not build it
Partner-reported blocking defects open at GA	Zero critical, fewer than 5 high	Distinguishes a controlled launch from an early one
Security reviews passed	At least 1 of 3 partners completing a full review	The diligence risk this programme exists to retire
Metering accuracy	Within 2%	Billing you cannot reconcile is worse than billing you cannot issue
Measured SLO attainment	30 continuous days meeting published targets	Converts an availability claim into an availability record
Waitlist qualified leads	40 or more matching the Business profile	Evidence that horizon two has demand to build against

15.6 Closing statement

The proposition of this programme is narrow and testable. At week 24, CerebroHive will be able to put a customer's data behind a verified identity, run a multi-agent workflow that genuinely plans rather than replays a fixture, remember what it did, prove what it did to an auditor, and bill for it accurately. None of those six things is true today. All six are prerequisites for every other ambition in the constitution, and none of them requires a new product to be invented.

That is the whole argument. The architecture is already good enough to build on; the specifications are already written; the data model is already correct. What is missing is depth in a narrow vertical slice and the discipline not

to widen it. Twenty-four weeks, one slice, six gates, and a published record of what was actually true at each of them.

Appendix A — Glossary

Term	Definition
ADR	Architecture Decision Record. A dated, owned document stating a decision, the options rejected and the reasoning. Required for every structural change in this programme.
Cerebro applications	The business-facing product family: Studio, Flow, Agent, Search, Archive, Insight, ERP, CRM and the rest.
DAG	Directed acyclic graph. The form a planner emits: nodes of work with dependency edges and no cycles.
Definition of done	The programme-wide standard in Section 4.4. Not negotiable per item.
Design partner	An early customer under a contract that exchanges preferential terms for structured feedback and a security review.
Exit signal	The single observable fact that determines whether a week in Section 6 succeeded.
Hive Credits	The consumption unit in the commercial model, burned by inference, agent compute, vector storage and completed workflow tasks.
Hive platform	The developer and operator product family: Forge, Ops, API, Identity, Shield, Data, Vector, Govern and the rest.
HiveSwarm	The internal name for the Python, Go and TypeScript multi-agent orchestration system discovered in the polyglot audit.
Intelligence Mesh	The product vision: a unified intelligence layer connecting people, data, models, agents, workflows and infrastructure.
M10.1–M10.7	The seven phases of the agent runtime programme, from real execution through to the public SDK.
Platform Runtime	The TypeScript single-agent path: platform-api plus the agent-builder capability plus the AI gateway.
pgvector	The Postgres extension providing vector similarity search. Already installed and modelled at 1,536 dimensions.
RBAC	Role-based access control. Five organisation roles and roughly 28 permissions already exist in the auth package.
Scaffold hit	A regex match on a marker such as TODO, Mock or placeholder. A search heuristic, not a verdict.
System of record	The single authoritative implementation of a capability. Decision D3 names one for agent orchestration.
Tier 0–5	The five-layer architecture in Figure 1. Dependencies flow downward only.
WS0–WS6	The seven programme workstreams defined in Section 4.1.

Appendix B — Engineering Standards

B.1 Platform-wide required standards

Inherited by every product per the constitution. These are enforcement obligations, not aspirations, and every one of them is testable.

- **Authentication.** Every externally exposed endpoint verifies a token. No exceptions, including internal-only services, on defence-in-depth grounds.
- **Authorisation.** Role and permission checks are separate from authentication and are applied per resource, not per route family.
- **Audit.** Immutable logging keyed on the verified subject for every mutating action.
- **Events.** State changes publish to the event bus with a stable schema.
- **Observability.** An OTEL span per request hop; a metrics endpoint per service; alert rules for every declared failure mode.
- **API.** OpenAPI specification generated from source, published in CI.
- **Telemetry.** Usage tracked and attributable to a verified tenant. This is a commercial requirement, not only an operational one.
- **Documentation.** Generated where possible; updated in the same pull request where not.
- **Multi-tenancy.** Every query is tenant-scoped at the data layer, not at the route layer.
- **Explainability.** Every agent decision is traceable to the prompt, context and tools that produced it.

B.2 Code review standards

Change type	Reviewers	Additional requirement
Authentication, authorisation or tenancy	2, one from Security	A cross-tenant negative test in the same pull request
Prisma schema or migration	2, one from Platform	Migration rehearsed against a production-shaped dataset
Gateway, provider or routing	2, one from Runtime	Cost impact stated in the description
Infrastructure or Helm	2, one from Platform	Render diff attached
Public SDK surface (after MS5)	2, one Principal	Versioning impact stated; breaking changes require an ADR
Anything else	1	Definition of done met

B.3 Testing standards

- Unit tests cover the happy path and at least two failure modes.
- Integration tests exercise the real dependency. A test that passes against a mock proves the mock works.
- Every security-relevant change adds to the four-scenario suite: authenticated, unauthenticated, invalid token, cross-tenant.
- Evaluation tests gate merges on model-facing changes from week 10, with thresholds for hallucination, accuracy and safety.
- Load profiles run nightly against staging from week 16.

- Chaos experiments validate every declared fallback in the degradation map before MS4 closes.

Appendix C — Evidence Index

Every substantive claim in Section 3 traces to one of the following. This index exists so that any reader can verify the assessment independently rather than taking it on trust.

Claim	Source
Tenant identity read from client-supplied headers	`apps/platform-api/src/middleware/RequestContextMiddleware.ts` ; full trace in `audit/P0-AUTH-AUTHZ-GAP.md`
No auth library in platform-api's dependency tree	`apps/platform-api/package.json` ; confirmed in `audit/MILESTONE-25.5-PRODUCTION-READINESS.md`
forge-api uses the auth package correctly	`services/forge-api/src/auth/jwt.guard.ts`
Auth package contains real JWKS verification and RBAC	`packages/auth/src/jwt/verify.ts` , `packages/auth/src/rbac/permissions.ts`
AI gateway resilience is real	`packages/ai-gateway/src/gateway.ts` and the providers directory; `audit/RESILIENCE-AUDIT.md`
The AI gateway container has no HTTP server	`packages/ai-gateway/src/index.ts` is a pure barrel export; `packages/ai-gateway/Dockerfile`
Two Helm generations render into one release	`infra/helm/cerebro-hive/templates/` — service.yaml, deployment.yaml, rollout.yaml versus deployments.yaml; `audit/INFRA-RECONCILIATION-PLAN.md`
platform-api never receives the AI secret	Helm conditional traced in `audit/MILESTONE-25.5-PRODUCTION-READINESS.md`
The swarm planner is hard-coded	`services/swarm-runtime/src/Planner.ts` ; `audit/RESILIENCE-AUDIT.md`
The execution engine's worker provider is simulated	`services/swarm-runtime/src/ExecutionEngine.ts` — 800ms timeout and a magic failure string
PolicyEngine runs with zero registered policies	`apps/platform-api/src/server.ts` ; `audit/RESPONSIBILITY-MATRIX.md`
The RAG data layer is modelled and live	`packages/db/prisma/schema.prisma` ; migration `20260719202458_talent_os_architecture`
HiveSwarm is real and spans three languages	`audit/POLYGLOT-ARCHITECTURE-MAP.md`
Eight services are CI-built and Helm-absent	Cross-reference of values.yaml, both ArgoCD applications and docker-build.yml; `audit/MILESTONE-25.4C-RUNTIME-INTEGRATION.md`
Studio carries 421 flagged files of 1,163	`audit/scaffold-ranking.md`
Web readiness composite is 67 of 100	`audit/executive-dashboard.md`
Observability is real: five alerts, six panels	`infra/helm/cerebro-hive/templates/observability.yaml`
The M10.1–M10.7 phasing and its reuse analysis	`AGENT-RUNTIME-BACKLOG.md`
Five-tier model, boot order, degradation map	`CAPABILITY_ARCHITECTURE.md`
Editions, licensing philosophy, marketplace fee	`COMMERCIAL_STRATEGY.md`

Claim	Source
Vision, portfolio, maturity model, eight phases	CEREBROHIVE_CONSTITUTION.md

Appendix D — Assumptions Register

Every estimate in this document rests on the following. Each carries an owner and a validation date; an assumption that proves false triggers a re-baseline at the next gate rather than a silent absorption into contingency.

ID	Assumption	Impact if false	Validate by
A1	The live cluster's authentication posture matches what the repository implies	Incident response scope changes materially; R1 escalates immediately	Week 1 (D1)
A2	The Prisma schema requires no migration for conversations, memory or retrieval	Adds roughly 200 hours and pushes MS3 by one to two weeks	Week 2
A3	One Helm generation can be deleted without losing required behaviour	Reconciliation becomes a merge rather than a deletion; roughly 200 additional hours	Week 2 (D2)
A4	HiveSwarm and the platform runtime can converge without a rewrite of either	Convergence exceeds its 1,020-hour estimate materially; MS4 is at risk	Week 8
A5	The Studio audit finds remediable scaffolding rather than a required rewrite	Re-platform exceeds 1,180 hours; the facade compression option is exercised	Week 8 (re-baseline at MS2)
A6	A blended fully loaded engineering rate of $198 per hour is achievable	Payroll scales linearly; a 10% variance is roughly $327,000	Week 3 (D5)
A7	Nine permanent hires close within fourteen weeks	**The most commonly optimistic assumption in plans of this shape.** A four-to-six week slip pushes MS4 onward	Weeks 4, 8 and 14 by wave
A8	Inference costs approximate 340M tokens at current blended provider pricing	Direct effect on the $468,000 line; mitigated by routing by task class	Week 6 (first full cost review)
A9	Three suitable design partners can be contracted by week 8	The GTM track loses its feedback loop; MS6's proof points weaken	Week 5 (D6)
A10	Penetration testing yields no architectural finding requiring redesign	Remediation exceeds two cycles; MS6 slips	Week 20
A11	A velocity of 82 rising to 92 points per sprint is achievable at the planned ramp	Sprint 12's contingency is insufficient; scope compression is required	Continuous from sprint 2
A12	The eight CI-built, Helm-absent services are not carrying production traffic	The consolidation decision changes shape and the estate is larger than assumed	Week 6 (D4)

Appendix E — Decision Log Template

Every decision in Section 1.4, and every structural decision taken during the programme, is recorded in this form. The rejected-options section is the part that retains value: six months later, the question is almost never what was decided but why the alternative was not.

Field	Content
Identifier and title	ADR-nnnn, a short imperative title
Date and owner	The date the decision became binding and the individual accountable for it
Status	Proposed, accepted, superseded by ADR-nnnn, or rejected
Context	The situation requiring a decision, with evidence. Links to source files or audit reports where the claim is technical
Options considered	At least two, each with cost, risk and reversibility stated
Decision	What was chosen, in one sentence
Rejected options and why	**Mandatory.** An ADR without this section is a note, not a record
Consequences	What becomes easier, what becomes harder, and what becomes impossible
Reversal cost	What it would take to undo this later, stated honestly

End of document. Version 1.0, 28 July 2026. Prepared against the CerebroHive monorepo and audit series. Financial and schedule figures are planning estimates subject to Appendix D.